

Formation concepteur de Jeu vidéo

Documentation technique

Clément Lion

Nom du jeu : Run! Chicken Run!

Gaming Campus promotion Bogota

Date de soumission : Novembre 2025

Sommaire

A Introduction.....	4
1 But du rapport.....	4
2 Objectifs du projet.....	4
3 Gestion de la communication.....	5
B Definition du Projet.....	7
1 Extraits choisis du GDD.....	7
Overview.....	7
Concept.....	7
Setting.....	7
Piliers.....	7
Boucles de jeu.....	8
Difficulté/progression.....	9
Évolution du personnage.....	9
Récapitulatif du nombre de grains nécessaires pour évoluer.....	9
Aptitudes (cumulatives).....	9
Pouvoirs (Temporaires liés à la forme).....	10
Collectables.....	10
Obstacles.....	11
2 Scope final du prototype.....	11
3 Jouer au prototype.....	13
Controles.....	13
Progression.....	14
C Developpement du Projet.....	16
1 Planning des tâches.....	16
2 Organisation du code & des composants.....	21
3 Le joueur.....	25
Cahier des charges réduit.....	25
Démarche proposée.....	25
Programmation.....	27
4 Composants avec lesquels le joueur peut interagir.....	50
Cahier des charges réduit.....	50
Démarche proposée.....	50
Programmation.....	53
5 La génération procédurale de niveaux.....	60
Cadrage.....	60
Programmation.....	64
6 Créer les UIs (scores, boutons et gestion du HighScore).....	74
Menus.....	74
Éléments d'UIs.....	77
Enregistrement des HighScore.....	79
7 Design Pattern utilisés.....	82
Observer.....	82

State Machine.....	85
Singleton.....	90
8 Pratiques pour faciliter la lisibilité du code.....	93
D Conclusion.....	96
Informatique.....	96
Travail en équipe.....	96
Perspective.....	97
E Annexes.....	98
1 Procédure de test.....	99
Test des sauts.....	99
Test des événements.....	99
Test des aptitudes.....	102
Test des Chunks de niveaux.....	102
Tests papiers.....	104
Scène de test.....	104
PlayTest.....	106
Autres bugs repérés et corrigés.....	108
2 Suivi des échanges lors du projet.....	109
Réunions d'équipes.....	110
[TEMPLATE].....	110
Réunions.....	110
Date des séances d'échange avec un enseignant.....	118
Echantillon de captures d'écran échanges entre réunions	119

A Introduction

1 But du rapport

Ce dossier a pour but de présenter le troisième et dernier projet de la formation Développeur Jeu Vidéo de GamingCampus. **Le projet a eu lieu sur sept semaines et sa réalisation était accompagné de cours en programmation pour Unity.** Le projet a été réalisé en partenariat avec les élèves du cours de Game Design qui ont fourni le GDD ainsi que tous les éclaircissements nécessaires en cas de questions sur les mécaniques de jeu.

Le but de ce rapport est de servir de complément au code du jeu programmé. De fait, je vais présenter ma démarche pour programmer le jeu, les choix techniques pris et je vais passer en revue les composants principaux construits. Si vous désirez voir tous les scripts du code, le code complet et commenté est disponible en accompagnement de ce rapport.

La nature changeante des spécifications et le fait que j'apprenais Unity en parallèle de la programmation du projet est cause du fait que le projet n'a pas avancé aussi rapidement que je l'espérais. Le projet ayant fait l'objet de quelques refactorisations et changement, **tous les éléments prévus dans le GDD n'ont pas pu être programmés. Afin de pallier à ce manque, j'essaie dans ce rapport d'expliquer comment j'aurais mis en place les éléments manquants.** Quand l'occasion se présente, j'essaie aussi de préciser comment j'aurais amélioré certains éléments du code si j'en avais eu le temps.

Ce projet est accompagné de captures d'écran de scripts de code et d'éléments Unity qui ont été programmés dans le cadre du projet.

2 Objectifs du projet

Le projet décrit dans ce rapport avait pour objectif de donner une première expérience de travail en équipe à partir d'un GDD. Une équipe de game designer se chargeait du GDD décrivant un jeu de type infinite runner en 3D, les développeurs devaient individuellement construire un jeu respectant au mieux le GDD et proposer des retours concrets afin de pouvoir l'ajuster et améliorer l'expérience du joueur.

L'outil de travail (Unity) était imposé. Le jeu décrit par le GDD devait obéir aux contraintes suivantes :

- **Utilisation du moteur Unity.**
- **Jeu en 3D.**
- **Génération procédurale de niveaux à parcourir.**
- **Découpage en scènes.**

- **Caméra fixe derrière le joueur.**
- **Gameplay évolutif.**

Dans notre équipe, nous avons six game designers qui ont travaillé sur le GDD et le suivi du projet. Les quatre développeurs (dont moi) qui ont travaillé chacun sur leurs prototypes échangeaient de façon intermittente afin de pouvoir partager les avancées de notre travail respectif.

Le projet s'est déroulé en deux phases principales :

- La phase d'idéation qui a eu lieu du 4 Juillet à fin Août. Lors de cette phase, il fallait développer un jeu qui réponde au cahier des charges et les Game Designer devaient rédiger le GDD. Les développeurs étaient aussi impliqués dans cette étape afin de proposer des idées et commenter sur la faisabilité technique des mécaniques de jeu.
- La phase de programmation a eu lieu du 15 Septembre au 30 Octobre 2025 et se passait en parallèle du cours pour présenter l'outil Unity. Lors de cette phase, le GDD a eu l'occasion d'être débattu et modifié lors de réunions et d'échanges sur Discord.

Le GDD ainsi qu'un suivi des réunions et des discussions ayant eu lieu sur les deux étapes du projet est disponible dans le dossier Annexe fourni avec ce rapport.

3 Gestion de la communication

Le suivi détaillé des points et des questions est disponible en Annexe.

Durant toute la partie d'idéation et de rédaction du GDD, les développeurs ont été invités à proposer leurs idées et à donner leur avis sur le jeu à réaliser. Nous avons eu **trois sessions de brainstorming** pendant lesquelles nous avons pu faire le tri entre les différentes idées afin d'essayer de proposer quelque chose d'amusant et d'innovant. A l'issue de ces points de cadrage, les Game Designer ont poursuivi les échanges afin de pouvoir rédiger le GDD.

Lors de la programmation du projet, il y a eu **trois sessions avec un encadrant afin de pouvoir échanger sur les avancées du projet** et pouvoir discuter des deadlines / livrables et éléments à essayer de prioriser dans le jeu. Cela fut aussi l'occasion de pouvoir se coordonner au sein des groupes.

Le projet est un projet individuel : les développeurs ayant chacun leur jeu à réaliser. Néanmoins, comme **j'avais mon projet sur github, il était assez facile pour moi de pouvoir échanger et poser des questions sur mon code en pointant directement la ligne de code qui me posait problème sur le Github.**

Finalement, tout au long de l'étape de développement, mon groupe avait un **canal discord avec les game designers afin de pouvoir poser des questions sur des points litigieux du GDD.** En cas de questions « mineures » (soucis sur les variables proposées / chunks de niveaux proposés au joueur),

j'essayais de faire des propositions car il était compliqué pour les game designer de trancher sans pouvoir faire des tests approfondis sur le jeu.

B Definition du Projet

1 Extraits choisis du GDD

Overview

- **Titre** : Run Chicken Run!
- **Cible** : 7 à 20 ans, Casual gamer.
- **Plateforme** : Ordinateur puis mobile iOS et Android.
- **Genre** : Infinite Runner 3D
- **Moteur** : Unity
- **Player** : Jeu Solo avec options en ligne (leaderboard).
- **Rating** : 3+

Concept

Le joueur incarne un œuf de poule qui cherche à s'échapper du poulailler. Pour y arriver, il va être amené à évoluer au travers de plusieurs transformations lui prodiguant des aptitudes et pouvoirs spécifiques.

Setting

Le jeu se déroule dans un univers fantastique et loufoque, inspiré du réel mais où la suspension d'incrédulité est forte. (On joue un poulet avec des pouvoirs et de l'humour).

Piliers

Course infinie : Défilement infini avec changements de biomes et augmentation de la difficulté/vitesse. Augmentation de la difficulté (= vitesse de course) de manière linéaire/exponentielle capée, ce qui force le joueur à récupérer des grains afin de pouvoir évoluer et être plus fort pour tenir la cadence/franchir les obstacles.

Évolutif : Possibilité d'évoluer et de se transformer, permettant de débloquent de nouvelles aptitudes (qui se stackent) et un nouveau pouvoir (spécifiques à la forme actuelle). On évolue en récoltant un certain nombre de grains.

Avoir des pouvoirs : En passant à travers un pickup, on débloquent un pouvoir/bonus d'une durée limitée. Ce bonus est lié à la forme d'évolution courante. L'utilisation du bonus est limitée par un cooldown. Tant que l'on a un bonus activé, on ne trouve pas de pickup.

Boucles de jeu

MACRO

Survivre le plus longtemps possible / Parcourir la plus grande distance.

Objectif : Aller le plus loin possible

Challenge : Éviter les obstacles.

Récompense : Nouveau record.

MESO

Objectif : Faire évoluer le perso

Challenge : Collecter un certain nombre de grains.

Récompense : Nouvelle forme.

Objectif: Améliorer ses pouvoirs

Challenge : Collecter des grains.

Récompense : Améliorations des skills dans l'hub qui aide le plus loin possible.

MICRO

Objectif: Éviter les obstacles

Challenge: Choisir le bon couloir / Sauter / Utiliser un pouvoir

Récompense : Ne pas mourir

Objectif : Activer des pouvoirs

Challenge : Collecter un pickup.

Récompense : Débloquent un pouvoir d'une durée limitée.

Difficulté/progression

Le rythme s'accélère en continu de manière linéaire ou exponentielle capée.

Progression : Évolution des formes qui permet d'accumuler de nouvelles aptitudes et de débloquent des bonus différents.

Évolution du personnage

Formes : Œuf → Poussin → Poulet → Coq → Super Coq → Dinosaur.

Chaque évolution confère :

- Une aptitude permanente (qui se cumule).
- Un pouvoir temporaire spécifique à la forme du personnage actuelle

Récapitulatif du nombre de grains nécessaires pour évoluer

Grains Accumulés	Evolution du personnage
50	Poussin
100	Poulet
250	Coq
500	Super Coq
1000	Dinosaur

Aptitudes (cumulatives)

- **Œuf** :

Base : déplacement simple (gauche/droite/saut).

- **Poussin** :

Glissade rapide : permet de se faufiler sous les obstacles.

- **Poulet** :

Planer : en maintenant le saut, prolonge la durée en l'air, jusqu'à un maximum de 3s

- **Coq** :

Double saut : possibilité de sauter une deuxième fois en l'air (double saut).

- **Super Coq :**

Dash aérien horizontal : propulsion horizontale rapide en l'air, permet d'éviter ou franchir des grands trous (permet de se déplacer de gauche à droite même en plein saut)

- **Dinosaure :**

Invincibilité titanique : Permet d'ignorer les obstacles destructibles en les écrasant simplement quand on passe dessus.

Pouvoirs (Temporaires liés à la forme)

Durée d'effet 10s

- **Œuf :**

Bouclier Coquille : immunité à 1 obstacle, la coquille se fissure et disparaît.

- **Poussin :**

Ralenti temporel : ralentit brièvement la vitesse du jeu

- **Poulet :**

Jet d'œuf : lance 3 œufs explosifs pour détruire les obstacles devant soi.

- **Coq :**

Cri du Coq : onde sonore qui repousse obstacles destructibles de 2m et Immobilise les obstacles en mouvements.

- **Super Coq :**

Souffle de feu : crache du feu nettoyant la ligne devant lui de tous les obstacles destructibles sur quelques mètres

- **Dinosaure :**

Écrasement sismique : smash frontal qui pulvérise tous les obstacles destructibles visibles à l'écran

Collectables

Grains normaux et variantes (Grain = 1 / SuperGrain = 5)

Bonus universels (pickups)

Aimant à grains, Multiplicateur

Bonus (temporaires et pas liés à la forme (?)):

Aimant à grains : attire automatiquement les grains autour pendant 10 secondes.

Multiplicateur de grains : double les grains récoltés pendant 10 s.

Bonus rare (exemple: 1 sur 2 partie)

Obstacles

- **Destructible** (avec pouvoir approprié), obstacle physique de petite taille (peuvent être ignoré et écraser en forme Dinosauré) à modéré , les obstacle en mouvement sont considéré comme petit et destructible.
- **Immuable**, les trous de diverse tailles et les obstacles physiques de grandes tailles sont indestructibles

Immuable	Destructible Petit	Destructible modéré	En mouvement
Trou	Palissade bois	Troncs d'arbre	Animaux
Gros Rocher	Arbuste	Portillon	Tracteur / Voiture
Muret	Buisson	Rocher	Vent
Gros Arbre	Grillage usé		Feuilles mortes
Flaque de boue			
Souche d'arbre			

2 Scope final du prototype

La difficulté du GDD provient du fait qu'il y a beaucoup d'éléments à programmer et à ajuster. La plupart des variables / dimensions / paramètres ont été décidés alors que nous n'avions pas fait de test. Il est alors compliqué de savoir si les valeurs choisies étaient pertinentes. J'ai eu l'occasion d'échanger avec **les game designers qui m'ont conseillé d'expérimenter avec les valeurs et de choisir celles que je jugeais adapter.** Nous comptons reboucler à la fin du prototype et faire la synthèse des valeurs originelles et des changements proposés.

Du fait de la durée réduite pour construire le projet, je n'ai pas eu le temps d'implémenter toutes les fonctionnalités demandées initialement. **Il me paraissait important d'essayer de mettre en place tous les éléments uniques au jeu, quitte à les simplifier quelque peu.** Ci-dessous se trouvent plusieurs compromis faits pour le développement :

- N'ayant pas de designers dans l'équipe, nous n'avons pas pu rajouter les modèles 3D et animations nécessaires au jeu. Cet aspect nous a embêté car il a été mentionné plusieurs fois qu'il était important d'avoir une ambiance légère et humoristique. Cet aspect était difficile à calibrer en l'absence d'Assets. **Nous nous sommes mis d'accord sur le fait de rajouter des placeholders afin de pouvoir explorer ce sujet et commencer à mesurer la complexité de gérer des assets en plus.**

- Le GDD mentionne beaucoup d'obstacles différents et plusieurs environnements. **Les différences entre les obstacles étant principalement liées à leur dimensions / leurs propriétés, j'ai programmé une classe Obstacle qui peut être ajouté à tout type d'objets de types obstacles, qu'importe ses dimensions.** Dans la partie de ce rapport dédié aux obstacles, je mentionne comment ajouter différents types d'obstacles (par exemple des obstacles qui se déplacent).
- Le GDD mentionne plusieurs collectibles avec des pouvoirs spéciaux. Tous n'ont pas pu être implémentés. **Afin de pouvoir garantir de la variété, trois collectibles différents ont été créés, le grain, le super grain et un collectible pour permettre au joueur d'activer son pouvoir spécial.** Les valeurs des grains ont été ajustées pour pouvoir accélérer un peu le jeu lors des phases de tests.
- Tous les différents pouvoirs spéciaux n'ont pas pu être implémentés à temps. Il était néanmoins très important de pouvoir tester que le joueur est capable de les lancer et de passer d'un état à un autre. **J'ai donc essayé de faire en sorte d'implémenter au moins deux aptitudes et deux pouvoirs spéciaux afin de pouvoir tester le changement d'état.** Les états n'ayant pas de pouvoir spécial implémenté se voient associer un pouvoir spécial « par défaut ». Il ne me reste plus qu'à remplacer le pouvoir par le nouveau pouvoir quand j'en aurait le temps.
- Normalement le joueur n'est pas censé choisir quand utiliser son pouvoir spécial, celui-ci s'active automatiquement lorsqu'il collecte un power up particulier. Pour que les pouvoirs soient satisfaisant, il aurait fallu que le game design soit adapté à cet élément aléatoire et que les power ups n'arrivent qu'à des moments où les pouvoirs sont utiles. Du fait de la contrainte de temps et de notre inexpérience en game design, **j'ai proposé au game designer de faire en sorte que le joueur active lui-même son pouvoir spécial quand il le juge opportun. Afin de pouvoir s'accommoder de ce changement, il est indiqué à l'écran combien de fois il peut encore activer son pouvoir spécial (entouré en rouge) et un collectible spécial vert a été ajouté pour permettre au joueur de récupérer une charge de son pouvoir spécial** (capture d'écran ci-dessous). Ce changement a aussi permis de pouvoir tester plus facilement l'utilisation des pouvoirs car nous étions dès lors capable de les activer en toutes circonstances.



Figure 1: Le nombre de charges de pouvoir spécial est indiqué à l'écran. Un collectible spécial permet de récupérer une charge de pouvoir spécial s'il n'en reste plus au joueur.

3 Jouer au prototype

Ce rapport est fourni avec un prototype du jeu. Le jeu commence sur un écran principal à partir duquel vous pouvez lancer une partie ou quitter le jeu. L'écran principal indique aussi le plus grand score jamais atteint par le joueur. Si le joueur meurt au cours du jeu, il atteint un écran de Game Over à partir duquel il peut recommencer une partie ou quitter le jeu.

Controles

Le joueur peut contrôler le personnage avec les flèches directionnelles (il peut se déplacer de droite à gauche, le niveau défile tout seul). **Le joueur peut sauter en appuyant sur la barre d'espace et activer son pouvoir spécial en utilisant la touche entrée.** Le nombre de charge de pouvoir du joueur est indiqué à l'écran à gauche.

Le joueur peut collecter des grains (les cubes jaune) pour augmenter son score.

Condition de victoire : Pas de condition de victoire, le joueur doit juste cumuler le plus grand score possible. Notamment, le high score (score le plus élevé jamais atteint) est indiqué sur l'écran de départ et l'écran de game Over. Essayez de le battre !

Condition de défaite : Le joueur doit éviter les obstacles rouges. Tout contact avec un obstacle rouge fait perdre la partie. Si le joueur quitte la zone de jeu (la passerelle marron), il va tomber et mourir. Evitez de tomber !

Progression

En fonction du nombre de grains que le joueur a collecté, le joueur change d'états et débloque de nouveaux pouvoirs spéciaux et des aptitudes. Un pouvoir spécial remplace le précédent et nécessite une charge afin d'être activé, une aptitude est une capacité que le joueur peut activer à tout moment et qui vient enrichir son arsenal. Les pouvoirs spéciaux sont actifs pendant 10s, le joueur gagne une charge de pouvoir spécial à chaque fois qu'il change d'état. Il est aussi possible de récupérer une charge de pouvoir spécial en récupérant un cube vert.

A chaque changement d'état, le niveau s'accélère.

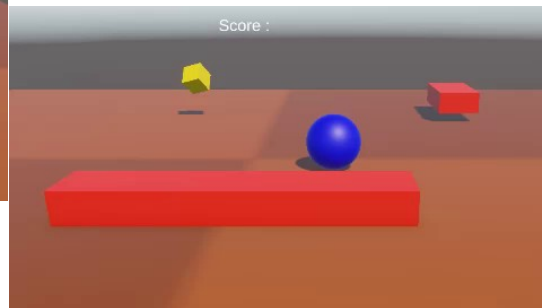
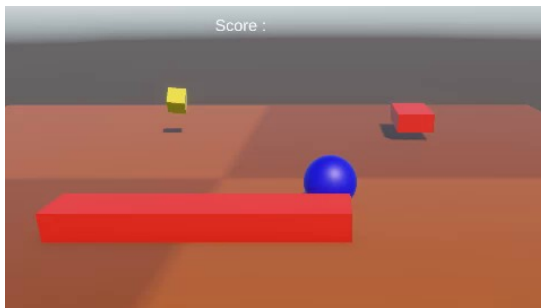
Grains collectés	Pouvoir Spéciaux	Aptitudes
0	Bouclier (Le joueur peut survivre à un obstacle)	N/A
50	Ralentissement (La vitesse du niveau est divisée par 2)	N/A
100	Ralentissement (La vitesse du niveau est divisée par 2)	Planer (En maintenant la barre d'espace, le joueur peut planer dans les airs jusqu'à 3 secondes)
250	Ralentissement (La vitesse du niveau est divisée par 2)	Double saut
500	Ralentissement (La vitesse du niveau est divisée par 2)	Dash (appuyer sur la touche 'e' et une direction pour faire un dash dans cette direction)
1000	Ralentissement (La vitesse du niveau est divisée par 2)	N/A

C Développement du Projet

1 Planning des tâches

Le jeu décrit à la partie précédente possède comme éléments différenciant le fait que le personnage principal peut changer de formes et que ces formes possèdent chacun une aptitude passive et un pouvoir spécial. **Il était important pour moi de pouvoir implémenter dans mon prototype au moins deux aptitudes et au moins deux pouvoirs afin que l'on puisse vérifier si cette particularité pouvait convaincre nos joueurs.** Afin de pouvoir me donner le temps de faire cela, j'ai essayé de répartir mes sept semaines de travail de la manière suivante :

- Semaine 1 et 2 : **Gestion du mouvement** → Dans une scène de test, création d'un personnage contrôlable afin de pouvoir programmer les mécaniques de déplacements et s'assurer que je suis capable de mettre en place un double saut, un glide ou encore un dash. Les avancées sont ensuite partagées aux game designer afin de clarifier certaines des valeurs proposées (exemple : hauteur de sauts). On peut voir ci-dessous deux vidéos prises à l'occasion de cette phase de tests afin de pouvoir tester comment le changement de paramètres influence le contrôle du joueur. **Livrable de fin d'étape** : une liste des paramètres pour le personnage principal.



Drawing 1: Ces vidéos présentent quelques essais avec des paramètres différents et des configurations de caméras différentes afin de s'assurer que le déplacement du joueur soit agréable et adapté au jeu.

- Semaines 3 à 5 : **Création de collectibles et d'obstacles, mise en place des mécanismes de génération automatique de niveaux d'infinite runners en utilisant des chunks.** Livrable de fin d'étape : Une première version du jeu où le personnage principal est capable de se déplacer, de collecter des collectibles et de rentrer en collision avec des obstacles. Le niveau est généré de façon procédurale à partir de chunks de niveau. **Livrable de fin d'étape** : Un premier prototype du jeu répondant à la description spécifiée afin de pouvoir tester comment les mécaniques de

jeu s'emboitent. On peut voir ci-dessous une capture d'écran présentant le rendu du jeu à ce stade là de développement :

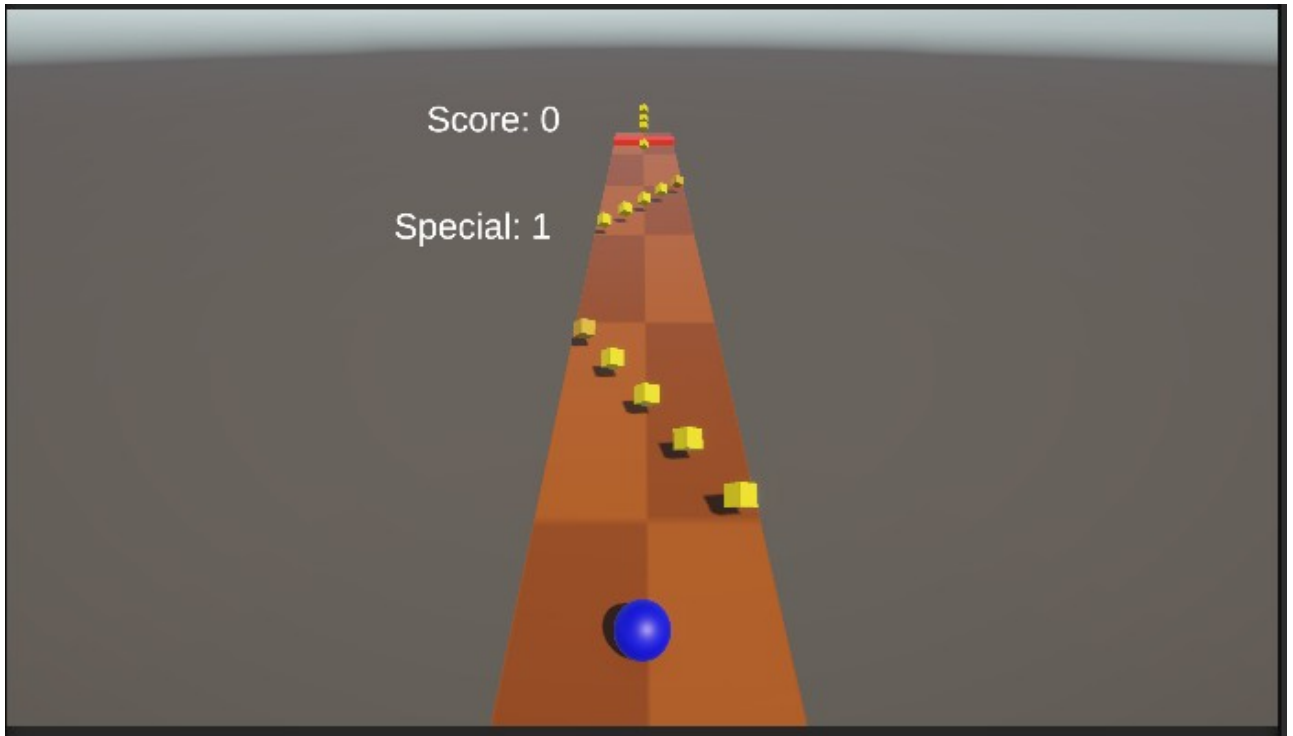


Figure 2: Un premier prototype n'inclut pas de modèles 3D, d'animations et de son mais permet de faire un test du jeu du début à la fin.

-
- Semaines 6 et 7 : **Ajout d'un menu de départ et de game over ainsi que d'un modèle 3D et de sons.** Le prototype utilisait jusqu'alors des formes géométriques afin de pouvoir représenter le joueur et les obstacles. Afin de pouvoir expérimenter avec des animations, les sons et les menus, j'ai essayé dans les dernières semaines d'ajouter ces éléments au jeu en utilisant des placeholders trouvés sur Mixamo (<https://www.mixamo.com/> modèles 3D et animations) ainsi que sur pixabay (<https://pixabay.com/sound-effects/> sons). **Livrable de fin d'étape :** Un prototype adapté qui inclus un menu de départ, un menu de game over, un joueur animé (animations incluses : marche, course, saut, animation de course spéciale quand le joueur active son pouvoir spécial) et des sons (une musique de fond et des bruitages qui se déclenchent quand des collectibles sont collectés). Ce prototype doit aussi enregistrer les high scores obtenus.



Figure 3: Menu de départ réalisé lors de la dernière phase de développement. Lorsque le joueur rentre dans le menu, le thème principal du jeu est joué. Il se repète en boucle. Le high score présenté est celui stocké par le jeu.

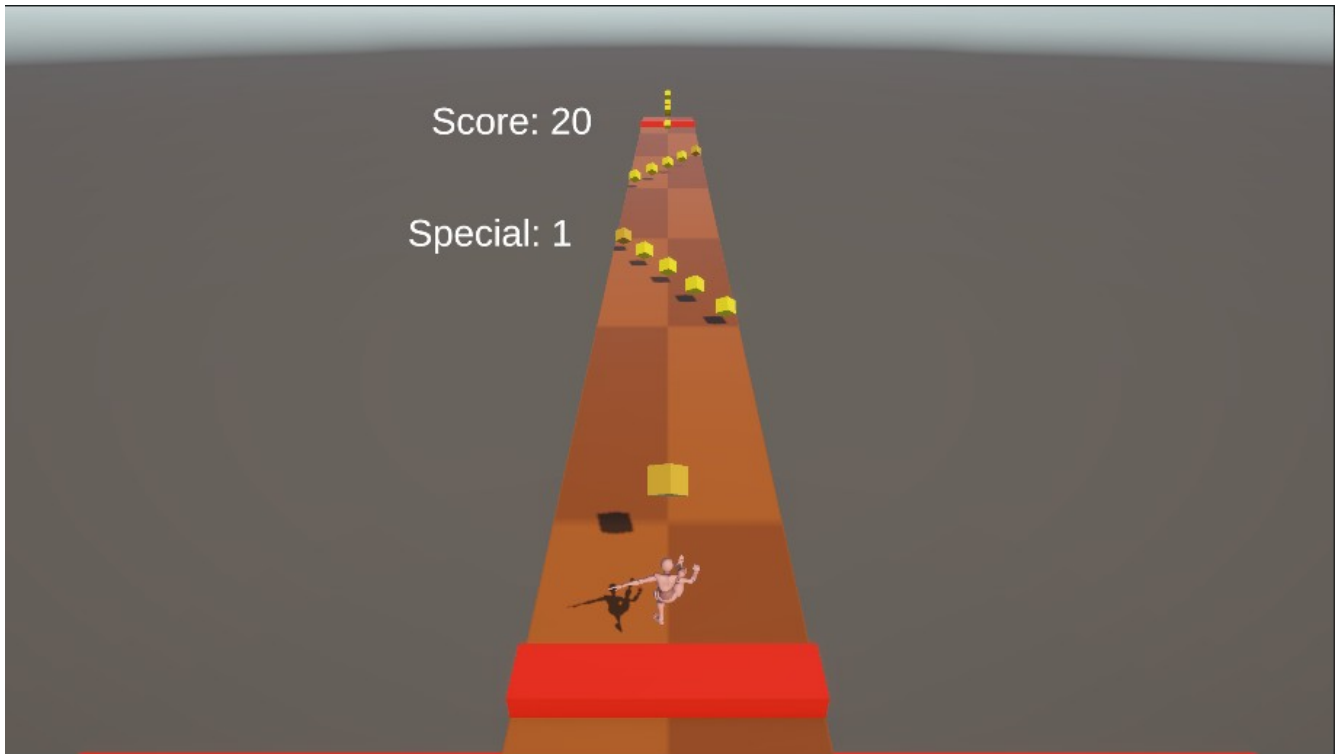


Figure 4: Ecran de jeu. Lors de cette étape de développement, j'ai rajouté un placeholder avec quelques animations (ici en saut) et j'ai rajouté un effet sonore lors de la récupération de collectibles.

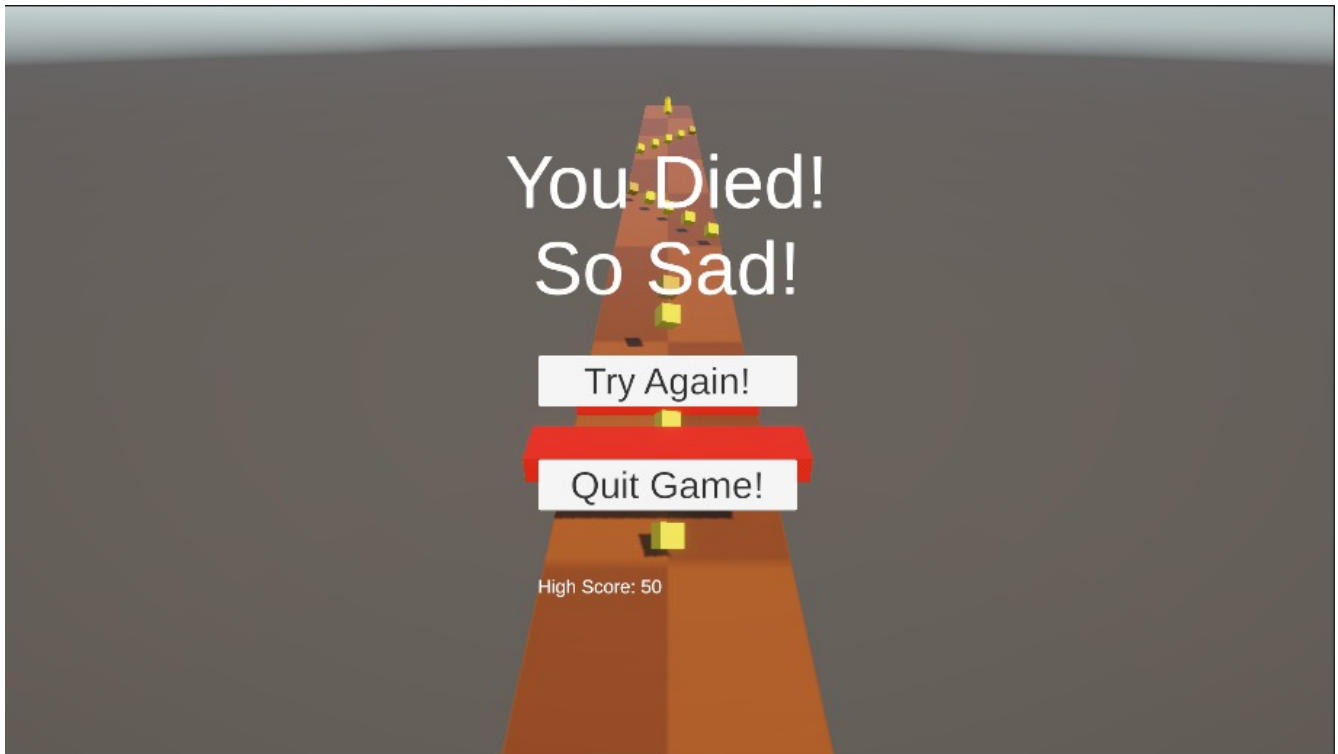


Figure 5: Ecran de Game Over du jeu. On voit le high score obtenu sur le jeu.

2 Organisation du code & des composants

Cette partie est une présentation exhaustive des scripts créés dans le cadre de ce projet. Elle est utile pour se repérer dans le projet. Si l'intégralité des scripts ne vous intéresse pas et que vous n'êtes intéressés que dans une mécanique particulière, sautez directement à la partie correspondante de ce rapport.

Les éléments du projet sont disponibles dans le dossier Assets. Voici la structure de dossiers adoptée :

- **Animations** : stocke les animations utilisées dans le jeu.
- **Data** : Stocke les informations à réutiliser entre deux sessions de jeu (dans notre cas le High Score).
- **Events** : Stocke tous les ScriptableObjects créés.
- **Materials** : Stocke les materials Unity.
- **Models** : Stocke les modèles 3D utilisés.
- **Resources** : Stocke les prefabs utilisés dans le jeu.
- **Scenes** : Stocke les scènes utilisées dans le jeu.
- **Scripts** : Stocke les scripts de code stockés dans le cadre du jeu.
- **Settings** : Dossier créé automatiquement par Unity pour stocker les paramètres.
- **Sounds** : Stocke les musiques et sons utilisés dans le jeu.
- **TextMesh Pro** : Dossier créé automatiquement pour permettre à Unity d'utiliser TextMesh Pro pour les Canvas.

Nous nous intéressons plus en détail au dossier Scripts et à la manière dont le code est organisé :

- **Camera** : (Ensemble des scripts liés au mouvement de cameras)
 - CameraControl.cs (Script pour que la caméra suive le joueur).
- **InfiniteRunnerComponents** (Scripts pour créer et mettre à jour les éléments de notre infinite runner ex : Chunks de niveaux, collectibles et obstacles)
 - Chunks : (Scripts pour gérer les Chunks)
 - ScriptableObjects (ScriptableObjects relatifs aux Chunks de niveaux)

- ChangeLevelSpeed.cs (ScriptableObject pour déclencher un Event pour changer la vitesse de défilement du niveau).
- ChunkCharacteristics.cs (ScriptableEvent pour rajouter des propriétés aux chunks de niveaux).
- ChunkDestroyer.cs (Classe pour détruire des morceaux de chunks).
- ChunkMover.cs (Classe pour faire défiler les morceaux de Chunks à l'écran).
- Chunk.cs (Classe pour définir ce qu'est un chunks et quels attributs et méthodes lui sont associés).
- Spawner.cs (Classe pour créer des Chunks de niveau).
- Collectibles (Scripts pour gérer les Collectibles)
 - ScriptableObjects (ScriptableObjects relatifs aux Collectibles)
 - ScoreEventChannel.cs (ScriptableObject pour déclarer une augmentation de score).
 - SpecialEventChannel.cs (ScriptableObject pour déclarer la récupération d'une charge de pouvoir spécial).
 - Collectible.cs (Classe parent pour les collectible).
 - Corn.cs (Collectible de base qui fait gagner des points).
 - CornMagnet.cs (Aimant à grain. Non Implémenté).
 - CornMultiplier.cs (Multiplicateur de grain. Non implémenté).
 - SpecialCorn.cs (Grain spécial qui permet de regagner des charges de pouvoir spécial).
 - SuperCorn.cs (Collectible de base qui fait gagner plus de points que le grain normal).
- Obstacles (Scripts pour gérer les Obstacles)
 - Obstacle.cs (Classe pour définir un obstacle, méthodes et attributs).
- Iactivatable.cs (Interface pour définir des objets qui peuvent être activés et désactivés si nécessaire).
- **Menus** (Scripts pour créer des menus et les éléments d'UI).
 - GameOverMenu.cs (Classe pour définir le menu de game Over).
 - HighScoreDisplay.cs (Classe pour afficher le high score obtenu dans les menus).
 - MainMenu.cs (Classe pour définir le menu de base pour commencer le jeu).

- **Player** (Scripts pour contrôler le joueur).
 - ScriptableObjects (ScriptableObjects relatifs au joueur).
 - Animation (ScriptableObjects pour déclencher des animations)
 - SpecialAnimationChannelSO.cs (ScriptableObject pour lancer l'animation d'activation d'un pouvoir spécial).
 - Input (ScriptableObjects invoqués lorsque le joueur appuie sur une touche).
 - DashEventChannelSO.cs (ScriptableObject invoqué quand le joueur appuie sur la touche de Dash).
 - JumpEventChannelSO.cs (ScriptableObject invoqué quand le joueur appuie sur la touche de saut).
 - MoveEventChannelSO.cs (ScriptableObject invoqué quand le joueur appuie sur les touches directionnelles).
 - SpecialEventChannelSO.cs (ScriptableObject invoqué quand le joueur appuie sur la touche de pouvoir spécial).
 - StatusEvolution (ScriptableObjects liés au changement de statut du joueur)
 - NoHealthEventChannelSO.cs (ScriptableObject invoqué quand le joueur ne possède plus de vie).
 - ObstacleCollisionEventChannelSO.cs (ScriptableObject invoqué quand le joueur rentre en collision avec un obstacle).
 - SetHealthEventChannelSO.cs (ScriptableObject invoqué pour modifier le nombre de vies du joueur).
 - StateChangeEventChannelSO.cs (ScriptableObject invoqué pour changer l'état actuel du joueur parmi la liste d'états possibles décrites plus tôt ex : poussin, coq...).
 - HighScoreEventChannelSO.cs (Un ScriptableObject utilisé pour déclarer un nouveau high score atteint).
 - States (Scripts liés aux différents états du joueur. Rappel des états : Œuf → Poussin → Poulet → Coq → Super Coq → Dinosaur.).
 - ChickenState.cs (Classe liée à l'état poulet).
 - ChickState.cs (Classe liée à l'état poussin).
 - DinosaurState.cs (Classe liée à l'état dinosaur.).

- EggState.cs (Classe liée à l'état oeuf).
- PlayerState.cs (Classe parent pour les états).
- PlayerStateMachine.cs (Machine à état qui gère la transition et les pouvoirs spéciaux des états).
- RoosterStateMachine.cs (Classe liée à l'état coq).
- SuperRoosterStateMachine.cs (Classe liée à l'état super coq).
- CollisionHandler.cs (Classe pour gérer les effets des collisions sur le joueur).
- Input.cs (Classe pour réceptionner les inputs du joueur et lancer des events pour partager l'information).
- PlayerAnimator.cs (Classe pour gérer les animations du joueur).
- PlayerController.cs (Classe pour gérer les déplacements du joueur et ses actions).
- PlayerHealth.cs (Classe pour gérer la vie du joueur).
- PlayerPowerUp.cs (Classe pour gérer les aptitudes du joueur).
- PlayerScore.cs (Classe pour gérer le score du joueur).
- **ScriptableObjects** (Scripts pour définir des classes particulières utilisant les scriptableObjects).
 - VoidEventChannel.cs (Classe parent pour définir des ScriptableObjects qui peuvent lever des events).
- **Services** (Services globaux au jeu).
 - Singleton (Mise en place du design pattern singleton)
 - Singleton.cs (Classe pour définir le design pattern singleton).
 - ImmortalSingleton (Singletons particuliers qui perdurent d'une scène à l'autre).
 - AudioManager.cs (Singleton qui gère les sons dans le jeu).
 - GameOverHandler.cs (Classe pour gérer les game Over).
 - SceneTransitionHandler.cs (Classe pour gérer les transitions entre scènes).
 - ScoreJsonWriter.cs (Classe pour lire des Json afin de stocker le high score du joueur).

3 Le joueur

Dans le cadre du développement de l'objet joueur, plusieurs design patterns sont utilisés. Leur fonctionnement exact est expliqué dans une partie plus loin. Dans la partie présente, nous allons les nommer et expliquer leur utilité mais sans aller plus avant dans le détail. Les éléments décrits dans cette partie sont consultables dans le dossier Assets/Player du projet.

Cahier des charges réduit

1. Le joueur est **contrôlable** par le joueur en utilisant le clavier.
2. Le joueur peut **interagir avec des collectibles et les obstacles**.
3. Le joueur **possède un état et change d'état** quand il passe des seuils de collectibles collectés.
4. Le joueur possède des **aptitudes passives qui se débloquent** au fur et à mesure du jeu.
5. **Le jeu doit s'arrêter quand le joueur meurt**.
6. Le joueur est **censé incarner un poulet**.

Ces différentes contraintes vont influencer la manière dont on va programmer notre joueur et les composants Unity que nous allons utiliser.

Démarche proposée

La difficulté liée au joueur est le fait qu'il doit gérer beaucoup de responsabilités. Afin de pouvoir travailler par incréments, j'ai décidé de faire en sorte de **dissocier autant que possible chacune des responsabilités et d'en faire une classe séparée**. Tous les composants dissociés sont ainsi groupés dans un empty qui représente le joueur. Ce joueur est ensuite stocké dans un **prefab** pour être facilement importable dans toutes les scènes.

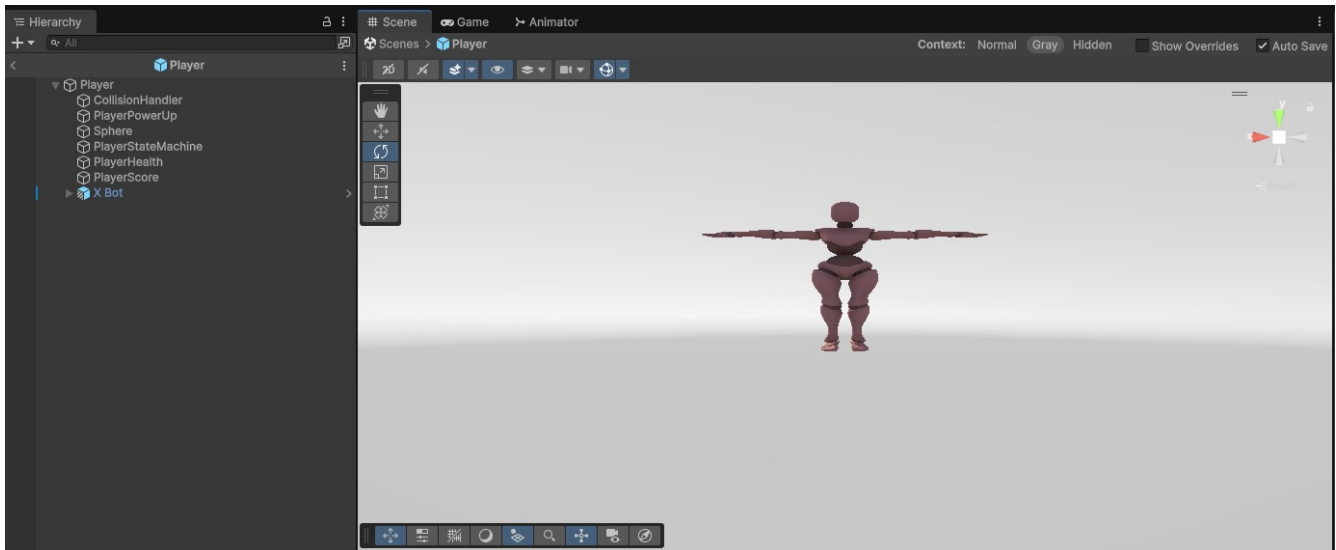


Figure 6: Le prefab joueur est composé de plusieurs scripts.

Décomposons chacun des éléments présents dans la capture d'écran :

- Nous possédons un **CollisionHandler** dont le but est de détecter si notre joueur est rentré en collision avec un autre composant et réagir en conséquence. Ce composant utilise le design pattern observer pour écouter les events de collision levés par les éléments de l'infinite runner.
- Nous possédons un **PlayerPowerUp** qui gère les aptitudes du joueur et les déverrouille au fur et à mesure que le joueur change de forme.
- Nous avons une **sphère** qui gère le mouvement du joueur et ses collisions (le joueur n'a une action directe que sur la sphère).
- Le **PlayerStateMachine** gère les différents états du joueur en utilisant le design pattern State Machine.
- Le **PlayerHealth** gère la santé du joueur. Il gère aussi le Game Over.
- Le **PlayerScore** garde une trace du score du joueur et communique un Event si jamais on passe les seuil définis pour changer d'état.
- Finalement, nous avons un **modèle 3D placeholder**. Le GDD précise que le joueur incarne un poulet. N'ayant pas trouvé d'asset correspondant, j'ai pris un placeholder afin de pouvoir étudier les difficultés liées aux assets et faciliter le remplacement d'asset quand j'en aurai trouvé un qui corresponde.

Programmation

Captation des Inputs

Un composant supplémentaire qui n'est pas associé au joueur mais qui est présent dans toutes les scènes est un script pour lire les inputs. **Le script d'input a pour but de dissocier les actions du joueur et l'objet physique joueur présent dans les niveaux.** Afin de faire cela, on utilise des Event Channels. Ces Event Channels sont tous des enfants d'une classe VoidEventChannelSO qui peut lever des événements. Tous les objets abonnés à l'Event Channel reçoivent l'événement et peuvent alors agir en conséquence. On parle du Design Pattern Observer. Plus de détails sur ce design pattern sont disponibles plus tard dans le rapport.

```
/*
A class to handle events with arguments.
*/

using UnityEngine;
using UnityEngine.Events;

[CreateAssetMenu(fileName = "VoidEventChannel_SO", menuName = "Events/VoidEventChannelSO")]
18 references
public class VoidEventChannelSO<T> : ScriptableObject
{
    35 references
    public UnityAction<T> onEventRaised;

    12 references
    public virtual void RaiseEvent(T value)
    {
        onEventRaised?.Invoke(value);
    }
}
```

Figure 7: Classe parente pour tous nos EventChannels. On possède une méthode pour lever un événement. Tous les composants qui écoutent l'événement peuvent alors agir en conséquence.

Notre script d'Input va s'appuyer sur quatre Event Channels différents afin de pouvoir **déclarer les entrées du joueur.** Les quatre Event Channels sont codés de la même manière. Ci-dessous se trouve l'Event Channel pour le mouvement :

```
/*  
Event to notify that the user has pressed the move button.  
*/  
  
using UnityEngine;  
  
[CreateAssetMenu(fileName = "MoveEventChannel_SO", menuName = "Events/MoveEventChannelSO")]  
2 references  
public class MoveEventChannelSO : VoidEventChannelSO<Vector2> { }
```

Figure 8: Cet EventChannel permet de communiquer comment le joueur souhaite se déplacer en deux dimensions.

Après avoir définir dans l'interface d'Unity les inputs actions pour les différentes actions de notre joueur, on peut ensuite les lire dans notre script d'input pour ensuite les communiquer via notre système d'Event Channels.

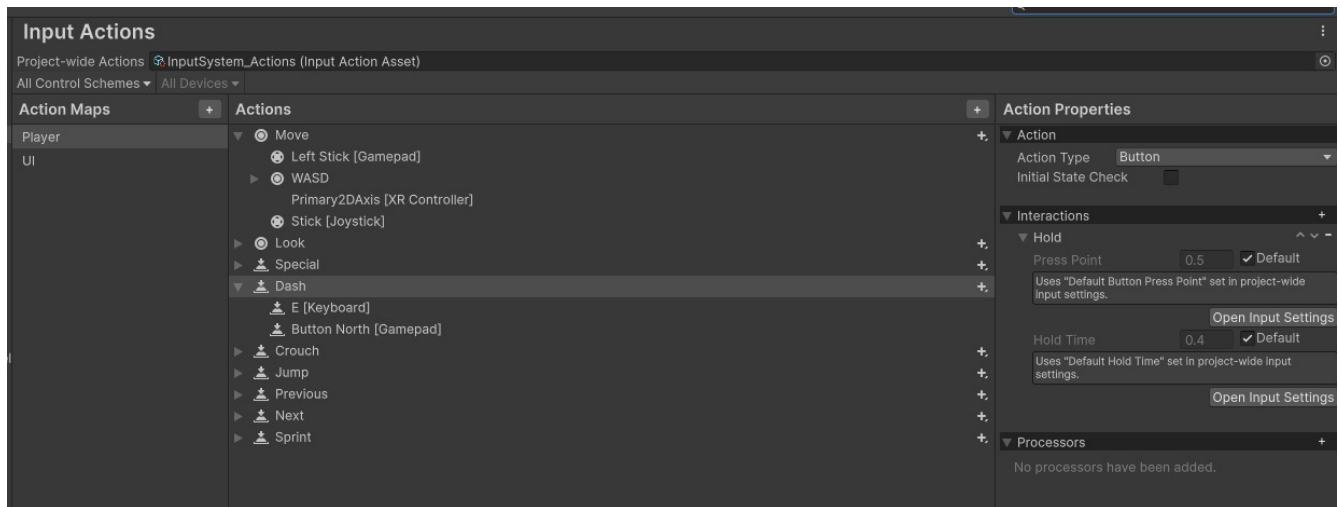


Figure 9: On définit dans Unity les Actions que l'on souhaite utiliser et on leur associe des touches.

```
/*  
A Script to handle player input. It uses Unity's input system.  
*/  
  
using UnityEngine;  
using UnityEngine.InputSystem;  
  
1 reference  
public class Input : Singleton<Input>  
{  
    #region Event Channels  
    [Header("Event Channels")]  
    [SerializeField]  
    1 reference  
    private MoveEventChannelSO _moveEventChannel;  
  
    [SerializeField]  
    1 reference  
    private JumpEventChannelSO _jumpEventChannel;  
  
    [SerializeField]  
    1 reference  
    private SpecialEventChannelSO _specialEventChannel;  
  
    [SerializeField]  
    1 reference  
    private DashEventChannelSO _dashEventChannel;  
    #endregion  
    /// <summary>  
    /// Reference to the actions our player will take.  
    /// </summary>  
    #region Actions  
    [Header("Movement")]  
    2 references  
    private InputAction _moveAction;  
    2 references  
    private InputAction _jumpAction;  
    2 references  
    private InputAction _specialAction;  
    2 references  
    private InputAction _dashAction;  
    #endregion
```

Figure 10: On récupère dans notre script ces actions et on se connecte aux Event Channels.

Le script d'Input lève à chaque Update un event pour chacune des actions pour indiquer son état. (Note : j'ai décidé de lancer un signal même pour une entrée utilisateur n'est saisie afin de vérifier si une touche est relâchée).

```
0 references
private void Start()
{
    _moveAction = InputSystem.actions.FindAction("Move");
    _jumpAction = InputSystem.actions.FindAction("Jump");
    _specialAction = InputSystem.actions.FindAction("Special");
    _dashAction = InputSystem.actions.FindAction("Dash");
}

0 references
private void Update()
{
    // Movement
    Vector2 moveValue = _moveAction.ReadValue<Vector2>();
    _moveEventChannel.RaiseEvent(moveValue);

    // Dash
    _jumpEventChannel.RaiseEvent(_jumpAction.IsPressed());

    // Special
    _specialEventChannel.RaiseEvent(_specialAction.IsPressed());

    // Dash
    _dashEventChannel.RaiseEvent(_dashAction.IsPressed());
}
```

Figure 11: L'input récupère les actions du joueur et lève des events qui correspondent à ces actions.

Ce script d'input va ainsi indiquer à chaque frame quel est l'état de toutes les actions du joueur, ce qui va nous permettre de jouer.

Mouvement

Les Input du joueur sont transformées en actions au niveau du script **PlayerController**. Ce script est attaché à une sphère invisible qui est contrôlée par le joueur. J'ai décidé de ne pas attacher mon script de mouvement au modèle 3D censé représenter le joueur d'une part car cela me facilitait le fait de changer d'asset et d'autre part car on peut vouloir que les hitbox et modèles soient à des endroits

différents. On verra un exemple de tels comportements plus tard avec les sauts. Le script PlayerController hérite des MonoBehaviour et on associe un Rigidbody à la sphère afin de prendre soin de la gravité. On lui ajoute aussi un Sphere Collider afin de gérer les collisions.

```
#region MonoBehaviour Methods
0 references | Unity Message
public void Awake()
{
    _rb = GetComponent<Rigidbody>();
}

/// <summary>
/// We subscribe to event channels to get user Input.
/// </summary>
0 references | Unity Message
private void OnEnable()
{
    _moveEventChannel.onEventRaised += Move;
    _jumpEventChannel.onEventRaised += Jump;
    _dashEventChannel.onEventRaised += Dash;
    _specialEventChannel.onEventRaised += Special;
}

0 references | Unity Message
private void OnDisable()
{
    _moveEventChannel.onEventRaised -= Move;
    _jumpEventChannel.onEventRaised -= Jump;
    _dashEventChannel.onEventRaised -= Dash;
    _specialEventChannel.onEventRaised -= Special;
}
```

Figure 12: A l'initialisation, on va récupérer tous les inputs et leur associer des fonctions pour chacune des actions possibles.

On veut tout d'abord déplacer le joueur. **Afin d'avoir un mouvement instantané (ce qui est essentiel dans un infiniteRunner), j'ai décidé de déplacer le joueur en changeant sa position.** Il n'y a pas d'accélération ou de décélération, le mouvement est constant. Mon objectif est de privilégier le contrôle : le joueur se déplace de façon constante et prévisible. Le mouvement se fait uniquement sur l'axe des x car le joueur ne peut se déplacer que sur ce plan. J'avais hésité à limiter le mouvement afin

d'éviter au joueur de tomber dans le vide s'il dépasse la zone de jeu. J'ai décidé de ne pas la faire afin de pouvoir permettre par la suite l'introduction de plateformes décalées ou d'autres obstacles similaires par la suite du jeu. Le script playerController signale au composant responsable de l'animation qu'il faudrait déplacer le modèle 3D via la méthode MoveModel.

```
0 references | Unity Message
private void FixedUpdate()
{
    // _rb.AddForce(_moveValue * _currentSpeed, ForceMode.Acceleration);
    transform.Translate(_moveValue * _currentSpeed * Time.deltaTime);
    if (Animator != null)
    {
        Animator.MoveModel(this.transform.position);
    }
}
#endregion

#region Movement
2 references
private void Move(Vector2 moveValue)
{
    _moveValue = new(moveValue.x, 0f, 0f);
    _moveValue.Normalize();
}
#endregion
```

Figure 13: Le mouvement se fait en changeant la position de l'objet afin d'avoir un mouvement instantané.

Saut

Le saut du joueur est plus piégeux à implémenter. En effet, **le joueur doit pouvoir débloquer un double saut et un glide qui lui permet de rester quelques secondes en l'air**. De même que pour les déplacements au sol, le saut va se faire en faisant une translation de notre joueur, cependant, contrairement au déplacement au sol, on va vouloir vérifier s'il y a un appui continu de la touche et si le joueur est dans les airs quand il veut faire un deuxième saut. **Important : pour le saut, on translate instantanément la sphère invisible auquel est attaché le collider. Le modèle 3D décrit un mouvement plus lent afin de ne pas surprendre le joueur**. On possède ainsi un moment où le collider et le modèle 3D ne sont pas au même endroit avant qu'elles ne se resynchronisent. Ceci n'est pas une erreur, j'ai fait ce choix car je voulais que les contrôles du joueurs répondent instantanément et

que le joueur puisse esquiver un obstacles jusqu’au dernier moment (d’où le déplacement instantané du Collider).

```
private void Jump(Boolean jumpAction)
{
    if (jumpAction)
    {
        // When the player presses the button, there is a brief window where he cannot jump.
        if (_canJump)
        {
            switch (_grounded)
            {
                case false:
                    if (
                        !_jumpContinuousPress
                        && _playerPowerUp.CanDoubleJump()
                        && _canDoubleJump
                    )
                    {
                        JumpTranslate();
                        _canDoubleJump = false;
                    }
                    break;
                case true:
                    if (!_jumpContinuousPress)
                    {
                        JumpTranslate();
                    }
                    break;
            }
        }
        // If the player can glide, he will glide by maintaining the jump button.
        // We make him glide by deleting any linearVelocity on the Y axis.
        if (_playerPowerUp.CanGlide())
        {
            _glideTimer += Time.deltaTime;
            if (_glideTimer < _glideValue)
            {
                rb.linearVelocity = new Vector3(
                    _rb.linearVelocity.x,
                    0f,
                    _rb.linearVelocity.z
                );
            }
        }
        _jumpContinuousPress = true;
    }
    else
    {
        _jumpContinuousPress = false;
    }
}
```

Figure 14: Pour sauter, on commence par vérifier si le joueur est au sol. Si ce n'est pas le cas, on vérifie s'il a débloqué le double saut. En fonction de la réponse on le fait sauter ou non. On vérifie ensuite si le joueur est en train de maintenir la touche de saut appuyée. Si c'est le cas et qu'il peut planer, on le fait planer en supprimant son mouvement de chute.

```
/// <summary>
/// A method to handle jump and / or double jump.
/// </summary>
2 references
private void JumpTranslate()
{
    // _rb.AddForce(Vector3.up * _jumpValue, ForceMode.Impulse);
    transform.Translate(Vector3.up * (_jumpValue + this.transform.position.y));
    StartCoroutine(MakeJumpAction());
}
```

Figure 15: Comme indiqué précédemment, le saut se fait en translatant le joueur dans les airs.

La vérification de si le joueur est au sol se fait via un raycast lancé en dessous du joueur qui vérifie si une plateforme de type « sol » se trouve immédiatement sous lui.

```
/// <summary>
/// We check if the player is grounded by shooting a raycast under him and seeing if it collides with the ground.
/// </summary>
/// <returns></returns>
2 references
private bool IsGrounded()
{
    Physics.Raycast(transform.position, Vector3.down, out RaycastHit hit, _raycastDistance);
    if (hit.collider is null)
    {
        return false;
    }
    if (hit.collider.CompareTag(_groundTag))
    {
        return true;
    }
    return false;
}
```

Figure 16: Vérification de si le joueur est à terre ou non.

Finalement, il y a un dernier élément dans le saut : le déclenchement des animations et le rétablissement depuis un saut. **Le rétablissement se fait via une coroutine qui désactive la possibilité de sauter pendant une période de cooldown. Cela permet d'éviter de déclencher par accident un double saut en gardant appuyé la touche de saut.** On lance l'animation de saut au moment où on fait un saut.

Au moment où le joueur touche le sol, on réinitialise toutes les variables liées au saut (possibilité de faire un double saut, possibilité de planer) et on met fin à l'animation de saut.

```
/// <summary>
/// A method to animate our 3D model and avoid automatic double jump when the key is pressed.
/// </summary>
/// <returns></returns>
1 reference
private IEnumerator MakeJumpAction()
{
    Animator.SetJump(true);
    _canJump = false;
    yield return new WaitForSeconds(_jumpCooldown);
    _canJump = true;
}

/// <summary>
/// A method to recover from a jump
/// TODO: jump recovery animation
/// </summary>
1 reference
private void JumpRecovery()
{
    Animator.SetJump(false);
    _canDoubleJump = true;
    _glideTimer = 0f;
    _canJump = true;
}
#endregion
```

Dash

Le dernier mouvement géré par le **CharacterController** est un **dash** qui va lui permettre d'éviter des obstacles en se déplaçant très rapidement sur le côté. Le dash peut se faire uniquement sur les côtés. On vérifie donc que le joueur a bien rentré une touche de direction et si c'est le cas, on le déplace très rapidement sur une courte distance. Je n'ai pas eu le temps de faire une animation pour le dash mais de même, je comptais faire en sorte que le collider se déplace très vite afin de garantir une esquive du joueur, le modèle 3D suit afin de faire en sorte que le mouvement ne soit pas trop inattendu.

```
#region Dash actions
/// <summary>
/// Logic for the player dash action. A dash is a sudden burst of speed from the player.
/// </summary>
2 references
private void Dash(Boolean dashAction)
{
    if (_canDash && _playerPowerUp.CanDash() && dashAction)
    {
        // We check that the player is actually moving and not standing still.
        if (_moveValue * _dashValue != _moveValue)
        {
            transform.Translate(_moveValue * _currentSpeed * _dashValue * Time.deltaTime);
            StartCoroutine(DashRecovery());
        }
    }
}

1 reference
private IEnumerator DashRecovery()
{
    _canDash = false;
    yield return new WaitForSeconds(_dashCooldown);
    _canDash = true;
}
#endregion
```

Figure 17: Le dash est un mouvement très rapide sur une courte distance. Pour éviter de lancer l'action en boucle, on possède une période de cooldown à la fin de l'action. Le dash ayant lieu sur les côtés, on vérifie que le joueur a bien rentré une direction.

Parenthèse : les aptitudes

On a eu l'occasion de voir l'implémentation du double saut, du vol plané et du dash. Ces mouvements sont des aptitudes du joueur, ils se débloquent au fur et à mesure du jeu quand le joueur change d'état. Les aptitudes disponibles du joueur sont stockées dans un Script appelé PlayerPowerUp.

```
/*  
An object to store important information about the player.  
It is a singleton meant to represent the player score, preferences and other.  
*/  
  
using UnityEngine;  
  
14 references  
public class PlayerPowerUp : MonoBehaviour  
{  
    [SerializeField]  
    0 references  
    private StateChangeEventChannelSO _stateChangeEvent;  
  
    /// <summary>  
    /// Since the player Abilities are cumulative, we switch them on when the user changes states.  
    /// </summary>  
    #region Abilities  
    11 references  
    public enum PlayerPowerEnum  
    {  
        2 references  
        Slide,  
        2 references  
        Glide,  
        2 references  
        DoubleJump,  
        2 references  
        Dash,  
        2 references  
        DestroySmallObjects,  
    }  
  
    [Header("Abilities")]  
    2 references  
    private bool _enableSlide = false;  
  
    [SerializeField]  
    2 references  
    private bool _enableDoubleJump = false;  
  
    [SerializeField]  
    2 references  
    private bool _enableGlide = false;  
  
    [SerializeField]  
    2 references  
    private bool _enableDash = false;  
    2 references  
    private bool _enableDestroySmallObstacles = false;  
    #endregion  
}
```

Figure 18: Le PlayerPowerUp conserve l'état actuel des aptitudes : est-ce que le joueur les a déjà débloquent ou non ? Afin de simplifier les tests, tous les champs sont des SerializeField pour pouvoir changer les états à la volée.

```
#region Getters and Setters
5 references
public void EnablePower(PlayerPowerEnum playerPower)
{
    switch (playerPower)
    {
        case PlayerPowerEnum.Slide:
            _enableSlide = true;
            break;
        case PlayerPowerEnum.Glide:
            _enableGlide = true;
            break;
        case PlayerPowerEnum.DoubleJump:
            _enableDoubleJump = true;
            break;
        case PlayerPowerEnum.Dash:
            _enableDash = true;
            break;
        case PlayerPowerEnum.DestroySmallObjects:
            _enableDestroySmallObstacles = true;
            break;
        default:
            break;
    }
}

0 references
public bool CanSlide()
{
    return _enableSlide;
}

1 reference
public bool CanDoubleJump()
{
    return _enableDoubleJump;
}

1 reference
public bool CanDash()
{
    return _enableDash;
}
```

Figure 19: On possède une méthode *EnablePower* pour débloquent les aptitudes, le reste sont des *Getters* afin de vérifier si oui ou non l'aptitude a été débloquée.

Gestion des pouvoirs spéciaux : une machine à états

Les pouvoirs spéciaux ne sont pas gérés par le CharacterController mais par la PlayerStateMachine. En effet, **comme les pouvoirs dépendent des états, j'ai voulu décorréler leur logique du CharacterController pour me donner le loisir de les changer sans incidence sur le déplacement.** Cela m'a permis de pouvoir tester les déplacements du joueur pendant les deux premières semaines du projet sans me soucier des pouvoirs spéciaux associés aux états. Chaque état est porteur de son pouvoir spécial. Le déblocage des aptitudes se fait lorsque le joueur rentre dans l'état. Afin de s'assurer que chacun des états possède les méthodes appropriées, j'ai créé une interface Istate qui définit les méthodes que doivent posséder chacun des états.

```
/*  
An Interface class in order to define states for our player.  
*/  
  
using System.Collections;  
  
/// <summary>  
/// Unity has its own StateMachineBehaviour states.  
/// In our case, we did not need the extra complexity.  
/// </summary>  
10 references  
public interface IState  
{  
    /// <summary>  
    /// A method to enter a state (change graphics for the state, play state sound...)  
    /// </summary>  
    7 references  
    public void Enter(PlayerStateMachine playerStateMachine, PlayerPowerUp playerPowerUp);  
    6 references  
    public void Update();  
    7 references  
    public IEnumerator Special();  
  
    /// <summary>  
    /// Method to Exit State  
    /// </summary>  
    7 references  
    public void Exit();  
}
```

Figure 20: Notre interface spécifie que les états doivent avoir une méthode Enter pour spécifier les aptitudes à débloquent quand on change d'état et éventuellement changer le modèle 3D. La méthode Special gère le pouvoir spécial. La méthode Exit est censé supprimer le modèle 3D de l'état duquel on sort.

Un exemple d'état est donné ci-dessous avec le ChickenState. On retrouve le fait que l'état débloque une aptitude lors du changement d'état et que l'état gère le déclenchement du pouvoir spécial.

```
/*
The ChickenState. it is the third state the player is in.
*/

using System.Collections;
using UnityEngine;

1 reference
public class ChickenState : IState
{
    5 references
    private PlayerStateMachine _playerStateMachine;
    1 reference
    private int _powerUpDuration = 10;

    2 references
    public void Enter(PlayerStateMachine playerStateMachine, PlayerPowerUp playerPowerUp)
    {
        playerPowerUp.EnablePower(PlayerPowerEnum.Glide);
        _playerStateMachine = playerStateMachine;
    }

    1 reference
    public void Update() { }

    /// <summary>
    /// Implement special effect here.
    /// Real effect not implemented here. We use the Chick's effect right now.
    /// </summary>
    /// <returns></returns>
    2 references
    public IEnumerator Special()
    {
        _playerStateMachine.SpecialAnimation(true);
        _playerStateMachine.InvokeChangeLevelSpeed(.5f);
        yield return new WaitForSeconds(_powerUpDuration);
        _playerStateMachine.InvokeChangeLevelSpeed(2f);
        _playerStateMachine.SpecialAnimation(false);
    }

    2 references
    public void Exit() { }
}
```

Figure 21: La forme standardisée des états permet de facilement en rajouter des nouveaux ou encore de modifier les propriétés d'un état sans que les autres soient impactés.

La transition des états et divers éléments communs aux états (comme le déclenchement de l'animation de pouvoir spécial) est géré par la machine à état. La machine à état déclenche les transitions et est l'interface entre l'état et le reste du jeu. (Note : dans notre cas, la machine à états possède beaucoup de responsabilités. Il doit être possible de la simplifier).

```
[Serializable]
24 references
public class PlayerStateMachine : MonoBehaviour
{
    #region State change
    17 references
    public enum PlayerState
    {
        1 reference
        EggState,
        2 references
        ChickState,
        2 references
        ChickenState,
        2 references
        RoosterState,
        2 references
        SuperRoosterState,
        2 references
        DinosaurState,
    }

    4 references
    public IState CurrentState { get; private set; }
    #endregion
}
```

Figure 22: La machine à état gère les états et stocke l'état actuel.

```
#region State transition
2 references
private void ConvertEnumToState(PlayerState state)
{
    IState nextState = null;
    switch (state)
    {
        case PlayerState.EggState:
            nextState = new EggState();
            break;
        case PlayerState.ChickState:
            nextState = new ChickState();
            break;
        case PlayerState.ChickenState:
            nextState = new ChickenState();
            break;
        case PlayerState.RoosterState:
            nextState = new RoosterState();
            break;
        case PlayerState.SuperRoosterState:
            nextState = new SuperRoosterState();
            break;
        case PlayerState.DinosaurState:
            nextState = new DinosaurState();
            break;
        default:
            break;
    }

    TransitionTo(nextState);
}

/// <summary>
/// Initializes with our first state.
/// </summary>
/// <param name="startingState"></param>
2 references
public void TransitionTo(IState nextState)
{
    CurrentState?.Exit();
    CurrentState = nextState;
    nextState?.Enter(this, _playerPowerUp);
    _specialCharge = 1;
}
#endregion
```

Figure 23: La machine à état gère les transitions entre les états.

```
/// <summary>
/// Each state has a different special capacity.
/// </summary>
1 reference
public void Special()
{
    if (CurrentState != null && _specialCharge > 0)
    {
        StartCoroutine(CurrentState.Special());
        _specialCharge--;
    }
}
```

Figure 24: La machine à état gère le déclenchement des pouvoirs spéciaux et le nombre de charges de pouvoirs spéciaux.

Collisions

L'ensemble des collisions avec le joueur est géré par un **CollisionHandler**. Le but est de **permettre une plus grande clarté dans l'interface Unity**. En effet, quand une collision a lieu, un Event est levé par un EventChannel. Cela nécessite de drag and drop dans l'interface Unity un ScriptableObject qui possède mon EventChannel. Afin de pouvoir voir en un coup d'oeil quels sont les éléments gérés pour les collisions, j'ai choisi d'avoir un composant qui gère toutes les collisions.



Figure 25: Les Event Channels à écouter sont modifiables au même endroit et directement dans l'interface d'Unity afin de permettre une meilleur lisibilité.

La logique des collisions est ensuite gérée par des classes séparées (la collision avec un objet est gérée par la classe **PlayerHealth** et la collision avec un Collectible par la classe **PlayerScore**. Le but du collisionHandler est d'être un Observer.

```
#region Subscribe to events
0 references
void OnEnable()
{
    collectableEventChannelSO.onEventRaised += HandleCollectible;
    ObstacleEventChannelSO.onEventRaised += HandleObstacle;
}

0 references
void OnDisable()
{
    collectableEventChannelSO.onEventRaised -= HandleCollectible;
    ObstacleEventChannelSO.onEventRaised -= HandleObstacle;
}
#endregion

#region Handle collisions with items
2 references
private void HandleObstacle(Obstacle.Resistance resistance)
{
    _playerHealth.CollisionWithObstacle(resistance);
}

2 references
private void HandleCollectible(int score)
{
    _playerScore.IncrementScore(score);
}
#endregion
```

Figure 26: Le CollisionHandler récupère les informations sur les collisions et relaie l'information des collisions avec les classes concernées.

Santé et score

Les composants de santé et de scores gère les valeurs numériques et déclenchent des événements particuliers quand des seuils sont passés (Game Over quand la santé devient 0 ou communiquer le score lors des Game Over afin de vérifier s'il s'agit d'un high score ou non).

Animations

Le dernier élément qui compose le joueur est un Animator qui permet de gérer les animations. Dans notre cas, on possède quatre animations : une animation de déplacement, une de saut, une de course et une pour les pouvoirs spéciaux. Quand on appuie sur la touche de saut, le joueur saute (transition depuis n'importe quel état). Je n'ai pas utilisé le bloc Any State car j'avais peur de conflits entre les états (l'animation spéciale est aussi sensée se déclencher depuis Any State, que faire lorsque le joueur active son pouvoir spécial en saut ?). Il me semblait plus lisible de gérer les transitions en reliant chaque animation à chaque autre animation.

Le passage dans l'état `Special_run` se fait via un booléen. Le passage de la marche à la course se fait quand une variable « speed » passe un seuil. Afin de pouvoir donner un rythme au jeu, la vitesse de l'animation de marche est de course est proportionnelle à cette variable speed.

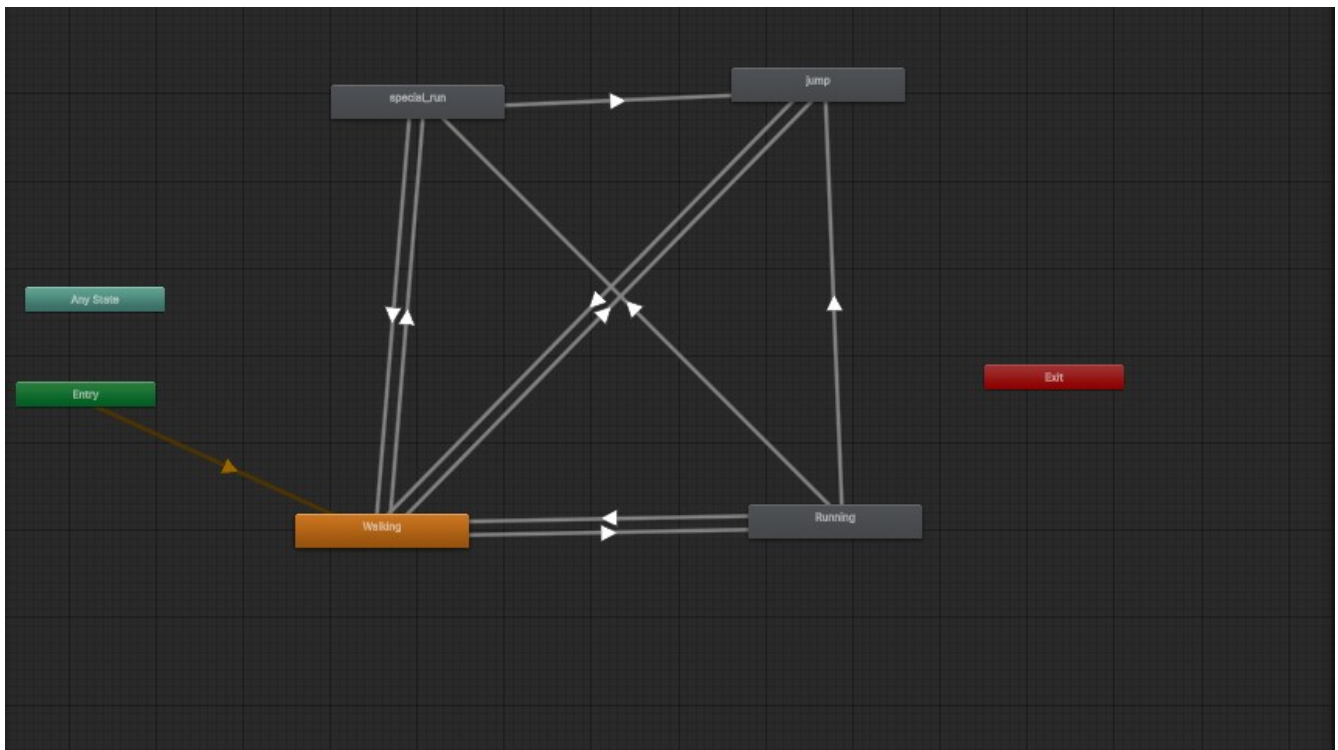


Figure 27: On possède quatre animations. Les animations de spécial et de saut se déclenchent lors d'actions particulières, les animations de marches et de courses bouclent.

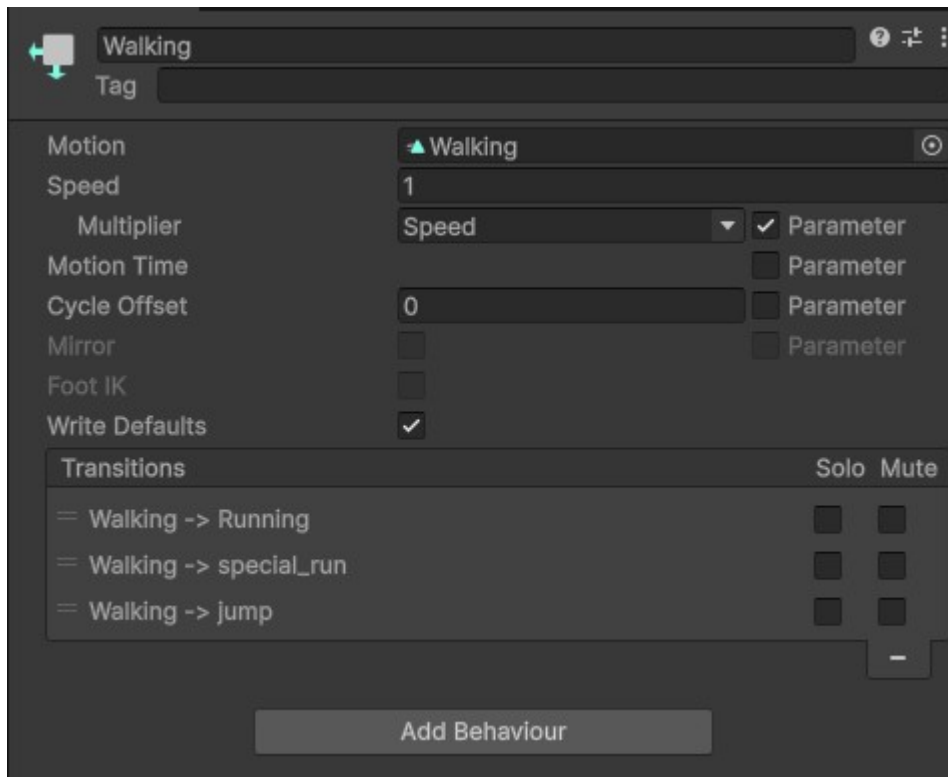


Figure 28: La vitesse des animations de déplacement est proportionnelle à une variable *Speed*.

4 Composants avec lesquels le joueur peut interagir

Les composants décrits dans cette partie sont consultables dans les dossiers Assets/InfiniteRunnerComponents/Collectibles et Assets/InfiniteRunnerComponents/Obstacles du projet.

Cahier des charges réduit

Le joueur interagit dans les niveaux avec deux types d'éléments :

- **Les collectibles donnent des effets positifs au joueur et augmentent son score.**
- **Les obstacles font perdre de la vie au joueur.**

Les collectibles comme les obstacles possèdent des effets cumulatifs (quand le joueur collecte suffisamment de collectibles et que son score passe un niveau, il change d'état) et leurs effets peuvent être modifiés (certains pouvoirs impliquent de détruire des obstacles).

Les collectibles doivent disparaître quand le joueur les collecte.

Certains pouvoirs peuvent désactiver les obstacles.

Il doit être facile de pouvoir ajouter de nouveaux obstacles et des nouveaux collectibles.

Démarche proposée

Afin de pouvoir construire les Obstacles comme les Collectibles, j'ai décidé de commencer par **une interface Iactivatable qui indique que les Obstacles comme les Collectibles peuvent être activés quand ils rentrent en jeu et désactivés quand nécessaire**. Pour les collectibles, j'ai commencé par une classe abstraite qui définit les mécaniques de base du composant. Les Obstacles n'ayant pas de complexité à l'heure actuelle, je me suis contenté d'une classe simple. Cette classe serait utilisée par la suite si on doit définir des obstacles spéciaux (exemple : des obstacles qui se déplacent).

Les Obstacles comme les Collectibles étant des éléments de jeu, ils doivent hériter des Monobehaviour. J'ai ensuite rajouté des règles particulières pour les collectibles sous la formes de scripts qui héritent des classes Collectibles. Les informations supplémentaires pourront être rajoutés via des ScriptableObjects. **Afin de pouvoir gérer la multitude d'effets que peuvent avoir les collectibles, j'ai décidé de dissocier la logique des collectes de collectibles de leurs effets : quand un collectible est ramassé, il va invoquer un Event et c'est à chaque autre objet d'agir en conséquence**. J'ai décidé de faire pareil pour les obstacles : leur unique responsabilité est de savoir si un élément rentre en contact avec eux.

J'ai rattaché les scripts à des cubes de tailles différentes avec des colliders de tailles différentes (les colliders des collectibles sont notamment plus grands que le cube afin de pouvoir

les rendre plus faciles à attraper). Afin de distinguer les collectibles et les obstacles, j'ai utilisé des matériaux différents en attendant d'avoir des assets intéressants à superposer.

Une fois que les grands types d'obstacles et de collectibles définis, j'ai créé un exemplaire de chacun des obstacles / collectibles que j'ai enregistré sous forme de **prefab** afin de pouvoir les réutiliser et les recréer facilement. Ci-dessous, voici plusieurs de prefabs créés.

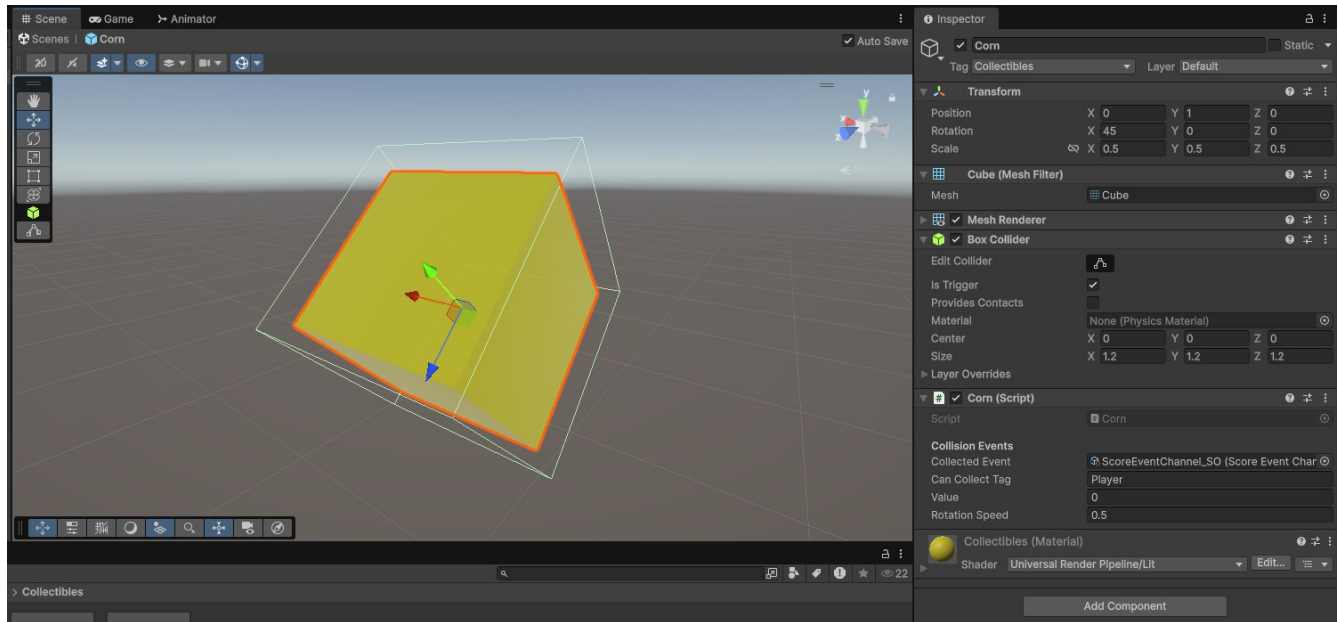


Figure 29: Prefab du collectible le plus basique : le grain. La logique du grain est géré par un script grain qui hérite de la classe abstraite collectible. Comme on peut le voir à l'écran, le collider du grain est légèrement plus grand que le cube afin que le joueur puisse l'attraper plus facilement.

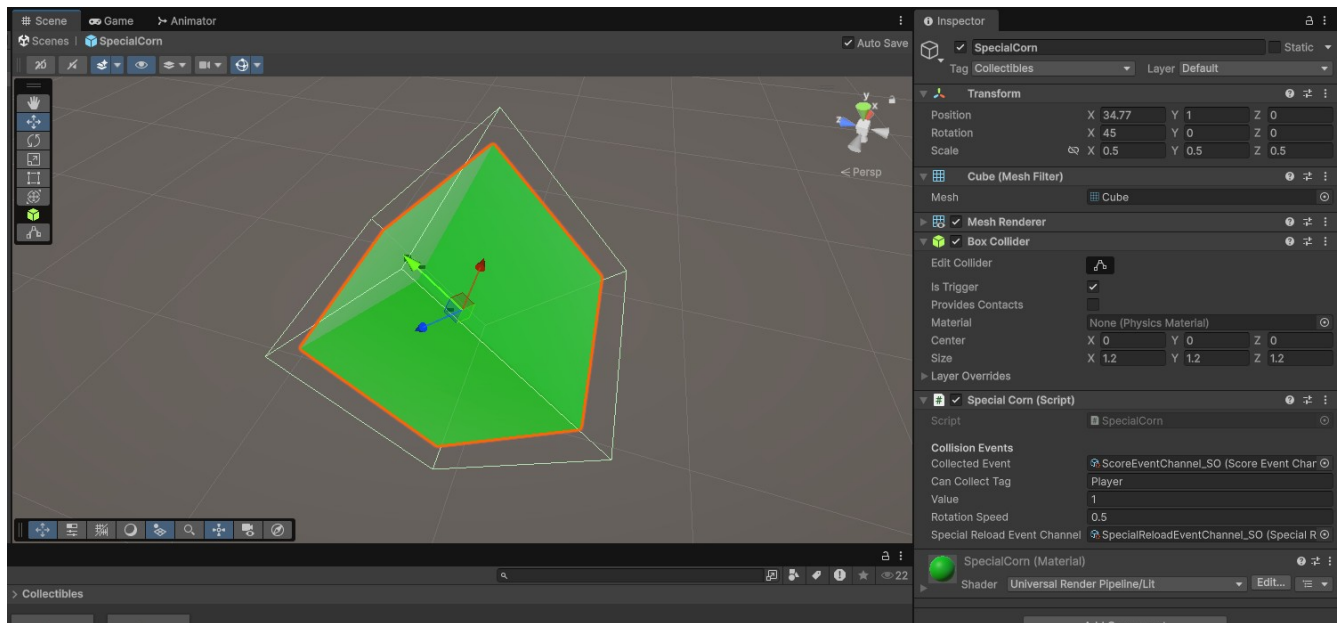


Figure 30: Prefab du Collectible SuperCorn qui permet au joueur de récupérer une charge de pouvoir spécial. Le collectible possède un script SpecialCorn qui hérite du script Collectibles.

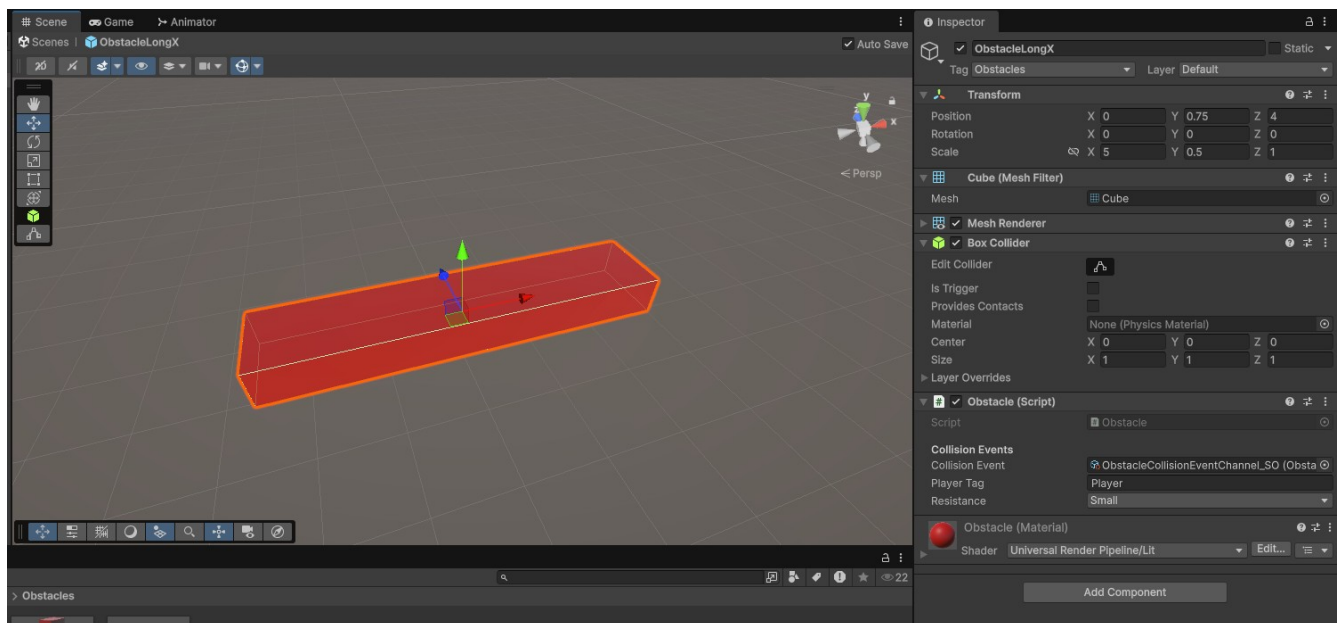


Figure 31: Prefab d'un obstacle. Il possède un script Obstacles qui lui dicte son comportement.

Programmation

L'interface IActivatable est l'élément de base. Il exige que tous les composants puissent être activés et désactivés. Cela nous sera utile pour pouvoir créer les éléments de notre infinite runner et les faire disparaître quand ils ne seront plus nécessaires :

```
/*  
An interface for objects that can be activated and deactivated.  
*/  
  
0 references  
public interface IActivatable  
{  
    0 references  
    void Activate();  
    0 references  
    void Deactivate();  
}
```

Figure 32: Une interface générique pour s'assurer que tous les éléments de notre infinite runner pourront être supprimés.

Les collectibles déclenchent un événement à chaque fois qu'il y a une collision avec un objet avec le Tag approprié (dans notre cas le Tag « Player », cette valeur est stockée dans une variable). Les collectibles doivent donc disposer d'un EventChannel (appelé _collectedEvent dans notre cas). Chaque Collectible possède une valeur en points (stocké dans _value) et il faut qu'on renseigne les méthodes de IActivatable. Cela donne le code ci-dessous pour notre classe abstraite Collectible :

```
/* Class to create collectible objects.*/

using UnityEngine;

[RequireComponent(typeof(Collider))]
5 references
public abstract class Collectible : MonoBehaviour, IActivatable
{
    /// <summary>
    /// Channel to declare Collected events (example: Score increase, other bonuses)
    /// </summary>
    [Header("Collision Events")]
    [SerializeField]
    1 reference
    protected VoidEventChannelSO<int> _collectedEvent;

    /// <summary>
    /// Tag to indicate who can collect the collectibles
    /// </summary>
    [SerializeField]
    1 reference
    private string _canCollectTag = "Player";

    [SerializeField]
    4 references
    protected int _value;

    #region Setters and Getters
    3 references
    public void Activate()
    {
        this.gameObject.SetActive(true);
    }

    3 references
    public void Deactivate()
    {
        this.gameObject.SetActive(false);
    }
    #endregion
}
```

Figure 33: Attributs de notre class Abstraite Collectible et méthodes des IActivatable.

```
0 references
void Awake()
{
    GetComponent<Collider>().isTrigger = true;
}

0 references
void Start()
{
    Activate();
}

6 references
public virtual void OnTriggerEnter(Collider other)
{
    if (other.CompareTag(_canCollectTag))
    {
        _collectedEvent?.RaiseEvent(_value);
        Deactivate();
    }
}
#endregion
```

Figure 34: Méthodes des Collectibles liées au fait qu'ils sont des monobehaviour.

Afin de voir comment cette classe nous aide à créer des collectibles, regardons l'exemple du code pour le collectible SpecialCorn qui permet au joueur de recharger une charge de pouvoir spécial quand il est touché. L'EventChannel est celui qui est associé à la recharge des pouvoirs spéciaux (appelé _specialReloadEventChannelSO). On associe à notre collectible une valeur. Finalement afin de pouvoir ajouter un peu d'attrait au tout, on ajoute une méthode Update qui va faire une rotation à notre Collectible. On peut de cette manière créer plein de collectible avec des effets variés (doubler la valeur des autres collectibles, gagner des valeurs en point différentes...) Il faut juste à chaque fois renseigner l'event Channel et la valeur appropriée.

```
/*  
A collectible to give the player another Special power Charge.  
*/  
  
using UnityEngine;  
  
0 references  
public class SpecialCorn : Collectible  
{  
    [SerializeField]  
    1 reference  
    private float _rotationSpeed = .5f;  
  
    [SerializeField]  
    1 reference  
    private SpecialReloadEventChannelSO _specialReloadEventChannel;  
  
    0 references  
    void Awake()  
    {  
        _value = 5;  
    }  
  
    0 references  
    void Start()  
    {  
        _collectedEvent = _specialReloadEventChannel;  
    }  
  
    0 references  
    void Update()  
    {  
        transform.Rotate(0, _rotationSpeed, 0);  
    }  
  
    4 references  
    public override void OnTriggerEnter(Collider other)  
    {  
        base.OnTriggerEnter(other);  
    }  
}
```

Figure 35: Afin de pouvoir programmer un collectible avec des pouvoirs spéciaux, on se contente de prendre notre classe abstraite collectible et de changer la valeur de l'event channel afin que le composant approprié qui gère le pouvoir du collectible est alerté de sa collecte.

Les obstacles fonctionnent de manière très similaires. La différence est dans le fait que chaque obstacle possède une résistance (les obstacles fragiles pouvant être détruits dans certaines formes). La résistance de l'obstacle est transportée par l'ObstacleCollisionEventChannelSO. **Comme nous en avons moins d'obstacle, je n'ai pas eu le besoin de créer de classe abstraite pour les représenter,**

néanmoins si on devait introduire des obstacles se déplaçant, il serait pratique de créer une classe abstraite obstacle.

```
/*  
A class to implement obstacles the player must avoid.  
*/  
  
using UnityEngine;  
  
4 references  
public class Obstacle : MonoBehaviour, IActivatable  
{  
    #region Collision properties  
    [Header("Collision Events")]  
    [SerializeField]  
    1 reference  
    private ObstacleCollisionEventChannelSO CollisionEvent;  
  
    [SerializeField]  
    1 reference  
    private string _playerTag = "Player";  
    #endregion  
  
    #region Obstacle properties  
    6 references  
    public enum Resistance  
    {  
        1 reference  
        small = 0,  
        0 references  
        large = 1,  
        0 references  
        unbreakable = 2,  
    }  
  
    [SerializeField]  
    1 reference  
    private Resistance _resistance = Resistance.small;  
    #endregion  
  
    #region Setters and Getters  
    2 references  
    public void Activate()  
    {  
        this.gameObject.SetActive(true);  
    }  
  
    2 references  
    public void Deactivate()  
    {  
        this.gameObject.SetActive(false);  
    }  
    #endregion
```

Figure 36: Attributs des obstacles et méthodes IActivatable. Les obstacles possèdent un EventChannel pour indiquer une collision et une résistance pour vérifier si le joueur ne casse pas l'obstacle.

```
#region MonoBehaviour methods
0 references
void Awake()
{
    GetComponent<Collider>().isTrigger = true;
}

0 references
private void OnTriggerEnter(Collider other)
{
    if (other.CompareTag(_playerTag))
    {
        CollisionEvent?.RaiseEvent(_resistance);
    }
}
#endregion
```

Figure 37: Méthodes monobehaviour pour indiquer comment les obstacles réagissent en jeu.

5 La génération procédurale de niveaux

Les éléments décrits sont consultables dans le dossier Assets/InfiniteRunnerComponents/Chunks du projet.

Cadrage

Je savais qu'il fallait que les niveaux soient générés aléatoirement. De cela découle quelques questions :

- **Comment faire en sorte que le joueur possède toujours un chemin pour avancer ?**
- **Comment faire pour tester / évaluer la difficulté du jeu et vérifier qu'on possède un challenge intéressant ?**
- **On doit générer un niveau infini, comment faire pour s'assurer que la mémoire de la machine ne sature pas ?**

Afin de répondre aux deux premières questions, j'ai décidé de faire la génération par chunks. Chaque tronçon de niveau est constitué d'une planche de sol, d'obstacles et de collectibles. Il est ainsi possible de vérifier individuellement pour chacun des tronçons que le joueur peut le parcourir et d'évaluer sa difficulté séparément. **Les chunks sont stockés sous forme de prefabs.** Le mécanisme de génération de niveaux peut alors choisir dans quel dossier de prefab il prend son prochain chunk et le génère. Afin de pouvoir gérer la difficulté, chaque chunk possède une chance d'apparition qui dépend de sa difficulté.

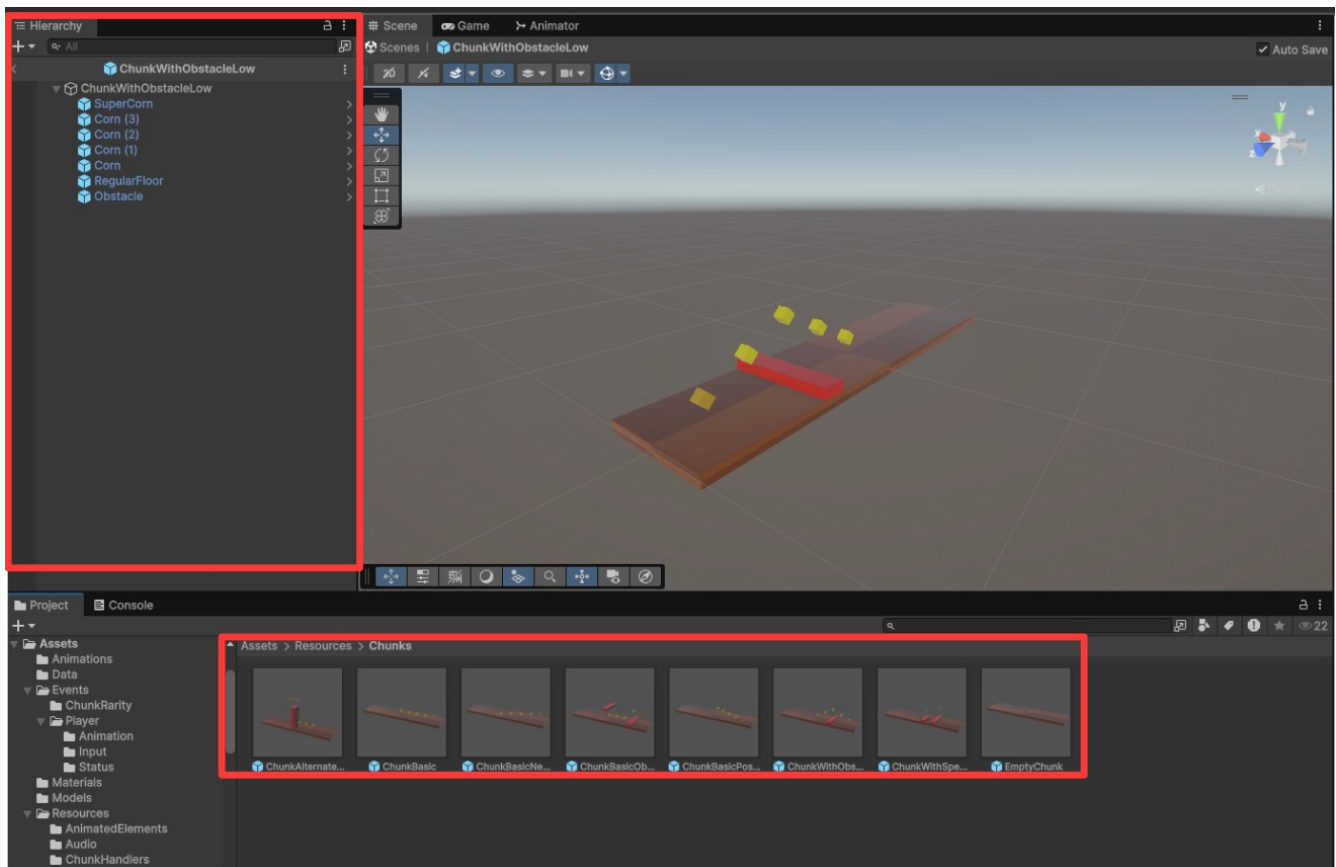


Figure 38: Un exemple de Chunk de niveau. Celui-ci est composé de sol, d'un obstacle et de cinq collectibles. En bas de l'écran, on peut voir l'ensemble des chunks possibles.

La rareté des chunks est précisé grâce à un ScriptableObject que l'on associe au chunk. Si on veut mettre en place des contrôles plus importants (exemple : certains chunks n'apparaissent que dans certaines zones), on peut pour cela rajouter un deuxième scriptable objet pour piloter cela.

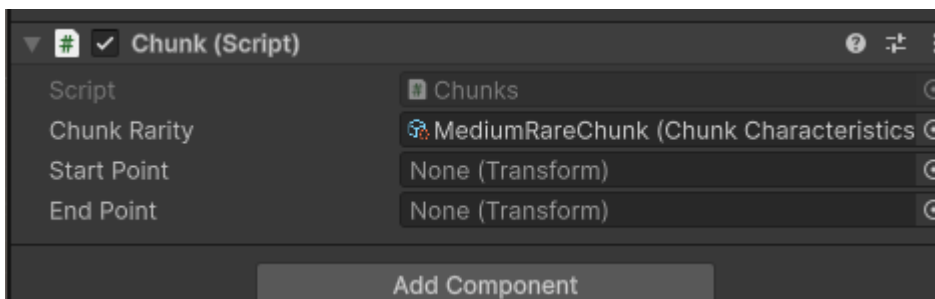


Figure 39: On définit la fréquence d'apparition du chunk grâce à un scriptable Object.

Le cahier des charges spécifie que les obstacles sont les mêmes pendant toute la partie pour une zone donnée. Après discussion avec les game designer, nous avons éclairci que **dès le début du jeu, il doit y avoir sur le chemin du joueur des chunks de niveau avec un chemin alternatif que le joueur ne peut pas encore franchir avec les aptitudes qu’il possède actuellement**. Cela est fait pour :

- **Susciter la curiosité chez le joueur.** Il voit des obstacles et se demande comment il va pouvoir les franchir.
- **Encourager la prise de risque.** Les obstacles alternatifs à franchir nécessitent une prise de risque mais rapportent potentiellement plus de récompense. Le joueur est ainsi encouragé à prouver sa maîtrise pour plus de récompense.

Je voulais prototyper cette idée. Pour cela, parmi les différents chunks de niveau possibles, deux d’entre eux possèdent un chemin alternatif qui nécessite une aptitude en particulier. Ces chunks de niveau particuliers sont présentés ci-dessous.



Figure 40: Le chunk de niveau ci-joint propose à droite un obstacle très grand qui ne peut être franchi que grâce au double saut. En récompense, le joueur a accès à trois super grains qui rapportent plus de points que les grains simples présents à gauche.

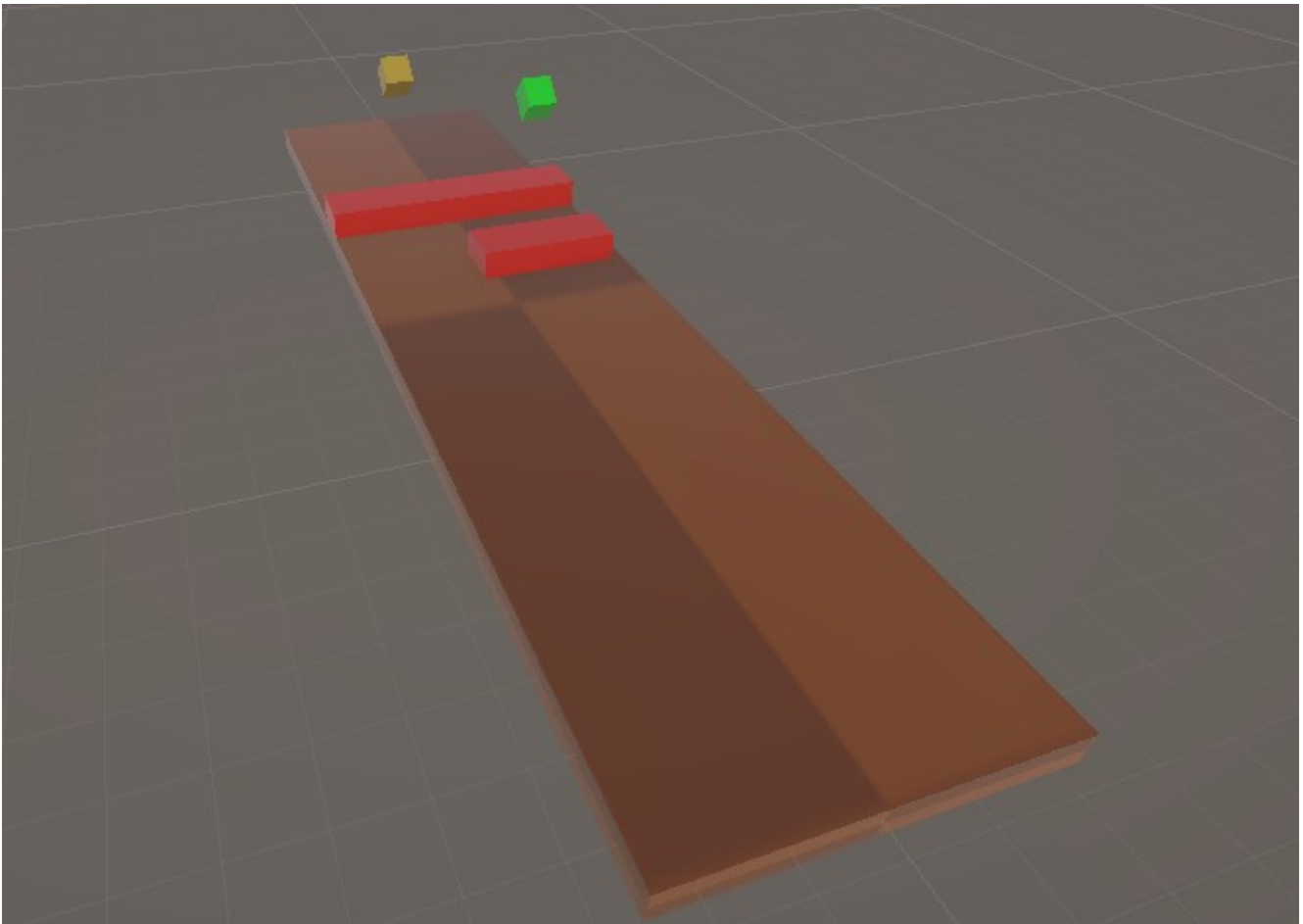


Figure 41: A droite se trouvent deux obstacles très rapprochés que le joueur ne peut passer que s'il possède le Glide pour planer au dessus. En récompense, il peut récupérer un collectible vert qui lui permet de regagner une charge de son pouvoir spécial. A gauche, le joueur n'a pas besoin d'aptitude pour passer mais ne récupère qu'un super grain.

Programmation

On va avoir trois composants qui vont gérer la génération de niveaux : **un Spawner gère la création des nouveaux chunks. Il possède une liste des chunks disponibles parmi lesquels il peut choisir quel chunk générer.** On possède un ChunkMover qui déplace les chunks dans la scène. C'est ce composant qui gère la vitesse de jeu. La vitesse augmente lors des changements d'états du joueur. Finalement, on possède un ChunkDestroyer qui va désactiver les chunks qui le traversent. On possède ainsi le cycle suivant :

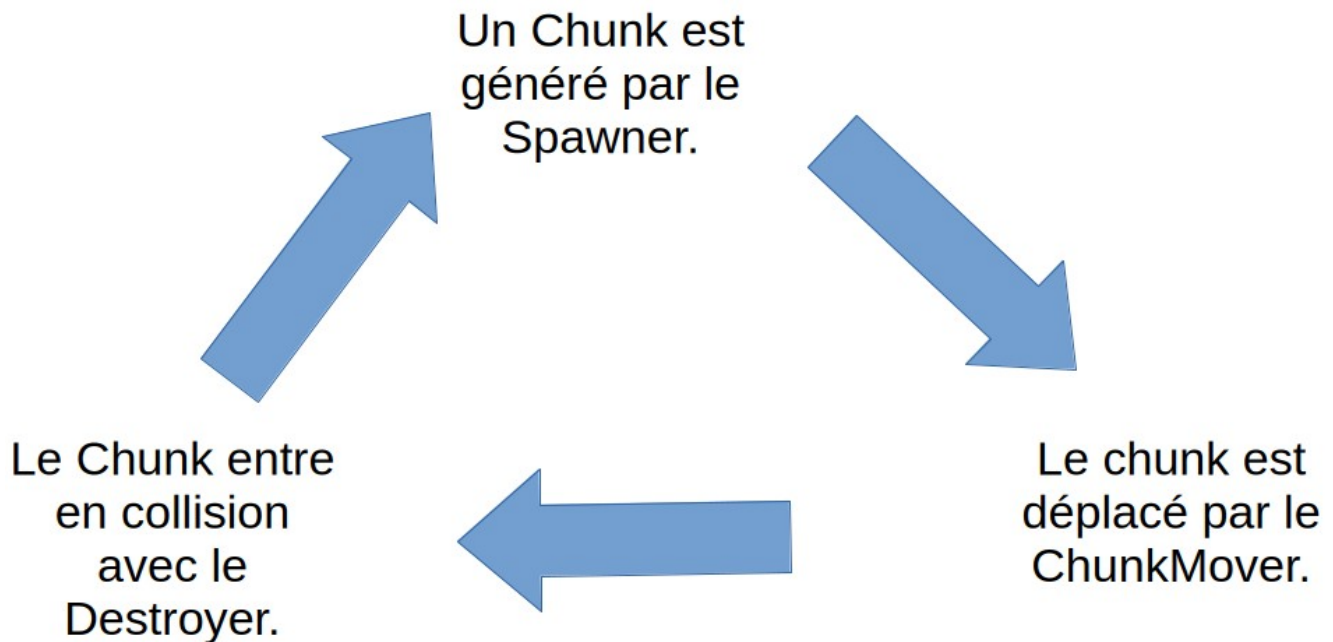


Figure 42: Les chunks sont créés par le Spawner et déplacés par le ChunkMover. Quand le Chunk sort de la zone de jeu, il est désactivé par le Destroyer. Le Spawner constate qu'un de ses chunks a été désactivé et en active donc un nouveau.

Les scripts de ChunkMover et ChunkDestroyer sont assez simples, je vais commencer par présenter ces deux scripts :

ChunkMover

Le ChunkMover possède une liste de Chunks actifs. Il va déplacer ces chunks dans la zone de jeu. S'il constate qu'un des chunks est désactivé, il va le retirer de la liste des chunks actifs et le signaler au Spawner qui devra en générer un nouveau en remplacement.

```
/// <summary>
/// We move all our chunks in the level.
/// If we find a disabled chunk, we remove it from the Active Chunks.
/// </summary>
0 references
public void Update()
{
    for (int i = _activeChunkList.Count - 1; i >= 0; i--)
    {
        Chunk myChunk = _activeChunkList[i];
        myChunk.transform.Translate(0, 0, -_levelSpeed * Time.deltaTime);
        if (myChunk.GetChunkState() == Chunk.ChunkState.disabled)
        {
            _activeChunkList.Remove(myChunk);
            int NumApparitions = myChunk.GetChunkRarity();
            if (NumApparitions > 0)
            {
                _spawner.EnqueueChunk(myChunk);
            }
        }
    }
}
```

Figure 43: Le *ChunkMover* déplace les chunks dans le niveau et vérifie que ses chunks ne sont pas désactivés.

Le *ChunkMover* est aussi responsable de la vitesse de défilement du niveau et possède des méthodes pour ajuster la vitesse. Ces méthodes sont déclenchées par les autres classes.

```
4 references
public class ChunkMover : Singleton<ChunkMover>
{
    [SerializeField]
    1 reference
    private Spawner _spawner;
    6 references
    private List<Chunk> __activeChunkList = new();

    #region Movement related properties
    [SerializeField]
    3 references
    private float _levelSpeed = 5f;

    [SerializeField]
    1 reference
    private float _speedUpFactor = 1.2f;

    /// <summary>
    /// We use the state change to know when to speed up the game.
    /// </summary>
    [SerializeField]
    2 references
    private StateChangeEventChannelSO _stateChangeEvent;

    [SerializeField]
    2 references
    private ChangeLevelSpeedSO _changeLevelSpeedChannelEvent;
    #endregion

    #region Getters and Setters
    2 references
    public void MultiplyLevelSpeed(float value)
    {
        _levelSpeed *= value;
    }

    2 references
    public void SpeedUpLevel(PlayerStateMachine.PlayerState value)
    {
        MultiplyLevelSpeed(_speedUpFactor);
    }

    2 references
    public void SpeedUpLevel(float value)
    {
        MultiplyLevelSpeed(value);
    }
}
```

Figure 44: Le ChunkMover est responsable de la vitesse de défilement du jeu.

ChunkDestroyer

Le **ChunkDestroyer** désactive les chunks avec qui il rentre en collision. Il possède pour cela un trigger collider qui déclenche la méthode de désactivation.

```
/*  
A class to destroy chunks that leave the screen.  
*/  
  
using UnityEngine;  
  
1 reference | Unity Script (1 asset reference)  
public class ChunkDestroyer : MonoBehaviour  
{  
    [SerializeField]  
    1 reference | Unity Serialized Field  
    private string _tagToDestroy = "Chunk";  
  
    0 references | Unity Message  
    void Start()  
    {  
        GetComponent<Collider>().isTrigger = true;  
    }  
  
    /// <summary>  
    /// The destroyer deactivates any chunk it touches.  
    /// </summary>  
    /// <param name="other"></param>  
    0 references | Unity Message  
    private void OnTriggerEnter(Collider other)  
    {  
        if (other.tag == _tagToDestroy)  
        {  
            other.GetComponent<Chunk>().Deactivate();  
        }  
    }  
}
```

Figure 45: Le *ChunkDestroyer* désactive les composants qui peuvent le croiser.

Spawner

Le Spawner est le composant clé de la génération des niveaux procéduraux. C'est lui qui va choisir quels chunks de niveau créer et ensuite les donner au ChunkMover pour qu'il les déplace.

Afin d'optimiser le code, j'ai utilisé de l'Object Pooling pour la création des niveaux : Dans notre cas, chaque chunk de niveau est constitué d'un sol, d'obstacles ainsi que de Collectibles. On ne peut pas grouper ces objets en un seul car nous avons besoin de pouvoir les désactiver individuellement si nécessaire (un collectible doit pouvoir être individuellement désactivé lorsqu'il est collecté sans pour autant que l'on désactive les autres collectibles). Le processus le plus exigeant pour la mémoire est la création de nouveaux objets. Comme on ne peut afficher de toute façon qu'un nombre limité de Chunks de niveau à l'écran, j'ai décidé qu'en début de jeu le Spawner va générer pour chacun des chunks de niveaux plusieurs exemplaires qui seront réutilisés. Le nombre exact est spécifié grâce à un ScriptableObject associé au chunk. Au cours du jeu, aucun nouvel exemplaire de chunk n'est créé, ce qui garantit que le nombre de chunks en jeu est constant et qu'on ne possède aucun pic d'activité lié à l'allocation mémoire.

Le code afin de pouvoir faire le pooling initial est disponible ci-dessous. On commence par lire l'intégralité des chunks disponible dans notre dossier de prefabs. On va instancier chacun de ces chunks plusieurs fois pour ensuite les charger dans une liste de Chunks disponible appelée _spawnQueue via la méthode EnqueueChunk. Le nombre d'instances à créer dans chaque cas est déterminé par une variable portée par un ScriptableObject associé au Chunks. Le but de cette démarche est que la probabilité d'apparition d'un chunks puisse être pilotée plus facilement via un ScriptableObject et pas directement dans le code.

```
/// <summary>
/// Loads all our available chunks and creates instances for each of our chunks depending on their rarity.
/// Every run starts with three empty chunks in order to let the player get the hang of things.
/// </summary>
1 reference
private void Initilialize()
{
    Object[] chunks = Resources.LoadAll(_chunkFolderName, typeof(GameObject));
    Object currentVal;
    Chunk myChunk;
    /// Initiliazes all the instances of our chunks.
    for (int i = 0; i < chunks.Count(); i++)
    {
        int NumApparitions = chunks[i].GetComponent<Chunk>().GetChunkRarity();
        for (int j = 0; j < NumApparitions; j++)
        {
            currentVal = Instantiate(chunks[i], new Vector3(0, 0, 0), Quaternion.identity);
            myChunk = currentVal.GetComponent<Chunk>();
            myChunk.Deactivate();
            EnqueueChunk(myChunk);
        }
    }

    /// Creates empty chunks at the beginning.
    Object beginningChunk = Resources.Load($"{_chunkFolderName}/{_beginningChunkName}");
    for (int i = 0; i < _numChunksAtBeginning; i++)
    {
        currentVal = Instantiate(beginningChunk, new Vector3(0, 0, 0), Quaternion.identity);
        myChunk = currentVal.GetComponent<Chunk>();
        AddChunkToActiveChunks(myChunk);
    }
}
```

Figure 46: Lors de la création de l'objet Spawner, on fait de l'object pooling afin d'éviter des ralentissements lors de la création d'objets supplémentaires au cours du jeu.

Finalement, les méthodes pour choisir activer un chunk et choisir aléatoirement le prochain chunk de niveau à générer sont définies ci-dessous :

```
#region Retrieve chunks from lists
2 references
private void AddChunkToActiveChunks(Chunk chunk)
{
    if (_chunkMover.GetNumberActiveChunks() == 0)
    {
        chunk.transform.position = new Vector3(
            SpawnPoint.position.x,
            SpawnPoint.position.y,
            Destroyer.position.z + _chunkSize
        );
    }
    else
    {
        chunk.transform.position = new Vector3(
            SpawnPoint.position.x,
            SpawnPoint.position.y,
            _chunkMover.GetLastChunkPosition().z + _chunkSize
        );
    }
    chunk.Activate();
    _chunkMover.AddChunk(chunk);
}

1 reference
private Chunk GetRandomChunk()
{
    int chunkPosition = Random.Range(0, _spawnQueue.Count);
    Chunk selectedChunk = _spawnQueue[chunkPosition];
    _spawnQueue.Remove(selectedChunk);
    return selectedChunk;
}

2 references
public void EnqueueChunk(Chunk chunk)
{
    _spawnQueue.Add(chunk);
}
#endregion
```

Figure 47: L'ajout de nouveaux chunks à la liste des chunks actifs se fait de façon aléatoire.

6 Créer les UIs (scores, boutons et gestion du HighScore)

Dans le cahier des charges, il est prévu que le jeu ait un écran d'accueil, un écran de Game Over et que le jeu sauvegarde le High Score du joueur. Les éléments présentés sont consultables dans le dossier Assets/Menus pour les menus et Assets/Services pour les services globaux (comme lecture de high score).

Menus

Le jeu possède deux menus différents : le menu de départ et le menu de GameOver. Dans ces écrans, l'utilisateur peut choisir de lancer une partie ou de quitter le jeu. Il voit aussi le high score affiché à l'écran. Afin de rendre l'écran plus intéressant à regarder, j'ai décidé de laisser en background les composants de génération de niveau décrits lors des parties précédentes.



Figure 48: Menu de départ du jeu, on possède deux boutons pour naviguer entre les scènes et le high score est affiché à l'écran. On voit en fond les chunks de niveaux défiler.

Afin de pouvoir générer le menu, on crée un objet Canvas auquel on ajoute deux boutons et deux zones de texte. Une des zones de texte est statique (et contient uniquement le « Welcome »), l'autre est dynamique et sert à afficher le HighScore actuel.

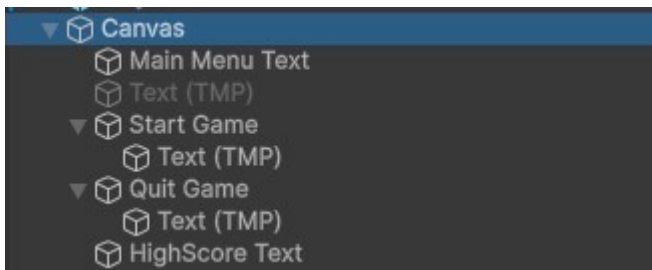


Figure 49: Composition de notre menu : un canvas possède deux éléments TMP pour écrire le "Welcome" et le score. On possède deux boutons pour naviguer dans le jeu.

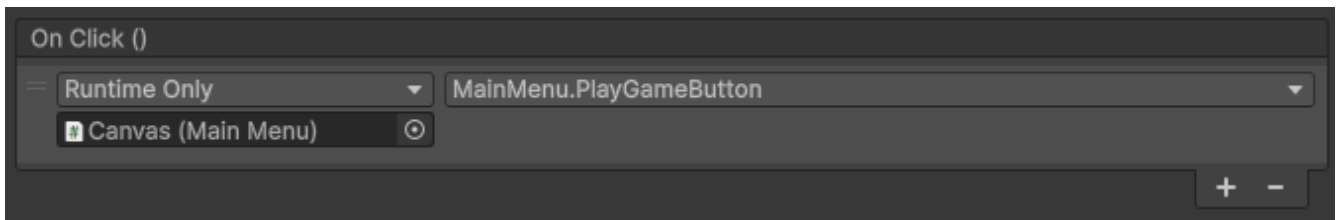


Figure 50: Chacun des boutons des menus exécute une des fonctions de notre Objet Menu.

Le code des boutons est fourni ci-dessous. Le joueur peut soit naviguer à la scène où a lieu notre infinite runner (appelée Level1Scene) ou quitter le jeu. Pour plus de flexibilité les menus passent par un SceneManager afin de gérer les changements de scènes. Cela est fait pour avoir plus de flexibilité et permettre éventuellement de rajouter des écrans de chargements et des effets visuels sur les transitions de scènes.

```
/*  
A simple menu to start or quit game.  
*/  
  
using UnityEngine;  
  
0 references  
public class MainMenu : MonoBehaviour  
{  
    1 reference  
    private SceneManager _sceneTransitionManager =>  
        ImmortalSingleton<SceneManager>.Instance;  
    ..  
    0 references  
    public void PlayGameButton()  
    {  
        _sceneTransitionManager.LoadScene("Level1Scene");  
    }  
  
    0 references  
    public void QuitButton()  
    {  
        // Quit Game  
        Application.Quit();  
    }  
}
```

Figure 51: Fonctions à exécuter quand les boutons du menu sont pressés. On peut au choix lancer le jeu ou quitter le jeu.

```
/*
A class to handle scene transitions.
*/

using UnityEngine.SceneManagement;

7 references
public class SceneManager : ImmortalSingleton<SceneManager>
{
    0 references
    private string _currentScene => SceneManager.GetActiveScene().name;

    3 references
    public void LoadScene(string scene)
    {
        SceneManager.LoadScene(scene);
    }
}
```

Figure 52: Le SceneManager est un singleton qui gère les transitions entre scènes. Le jeu n'ayant pas besoin de chargement, le composant reste assez simple. Il est à enrichir.

Note : Les codes des menus de départ et de Game Over sont identiques (bouton pour lancer le jeu et bouton pour quitter le jeu). **Cependant, j'ai jugé important de créer deux scripts différents car les deux menus doivent pouvoir changer de façon indépendante.** Je voulais ainsi permettre une ségrégation des interfaces.

Eléments d'UIs

Afin d'afficher le score que possède le joueur et les high scores dans les menus, j'utilise des composants TextMeshPro auquel j'attache un script monobehaviour dont le rôle est de stocker la variable à afficher et mettre à jour le texte de l'objet TMP au fur et à mesure du jeu.

Dans l'exemple ci-dessous, on possède une classe pour afficher le high score du joueur. Le composant est attaché à un composant TMP. Il va récupérer le high score auprès d'un composant qui possède l'information puis met à jour la valeur à l'écran. Dans notre cas, ce composant est accroché à un menu, le high score courant est donc une variable qui ne va pas changer dans la scène, on n'a pas besoin de mettre à jour la valeur dans la scène.

```

/*
A class to retrieve the user high score and display it in a text box.
This component must be attached to a TMP Component.
*/

using System.Collections;
using TMPro;
using UnityEngine;

0 references
public class HighScoreDisplay : MonoBehaviour
{
    3 references
    private TMP_Text _highScoreText;
    2 references
    private ScoreJsonWriter _jsonScoreWriter => ImmortalSingleton<ScoreJsonWriter>.Instance;

    /// <summary>
    /// This component must be attached to a TMP_Text component.
    /// Score is handled by a ScoreJsonWriter. We wait for it to be created to move on.
    /// </summary>
    #region MonoBehaviour methods
    0 references
    void Awake()
    {
        _highScoreText = GetComponent<TMP_Text>();
        if (_highScoreText == null)
        {
            Debug.LogError("HighScoreDisplay requires a TMP_Text component.");
            enabled = false;
        }
        StartCoroutine(WriteHighScoreText());
    }
    #endregion

    1 reference
    private IEnumerator WriteHighScoreText()
    {
        yield return new WaitUntil(() => _jsonScoreWriter != null);
        int score = _jsonScoreWriter.GetHighScore();
        _highScoreText.text = $"High Score: {score}";
    }
}

```

Figure 53: Classe servant à mettre à jour la valeur du high score à l'écran.

Dans les autres cas où on doit mettre à jour une valeur à l'écran, on met à jour la valeur lors de la réception d'un event ou alors on peut utiliser la méthode Update (exemple ci-dessous issu du composant PlayerScore évoqué lors d'une des parties précédentes).

```
0 references
void Update()
{
    if (ScoreText != null)
    {
        ScoreText.text = $"Score: {_playerScore}";
    }
}
```

Figure 54: Le score pouvant changé à chaque frame, on le met à jour dans la fonction Update.

Enregistrement des HighScore

Choix technique : Je souhaitais que le high score puisse perdurer d’une session de jeu à l’autre. J’ai décidé d’enregistrer le high score sous la forme d’un JSON qui contient la valeur du plus grand score atteint jusqu’alors :

```
Assets > Data > {} highscore.json > ...
1 [{"HighScore":40}]
```

Figure 55: Le high score est sauvegardé sous la forme d’un JSON de texte.

Ce système est extrêmement simple à traiter et à mettre à jour. Il suffit d’écraser le fichier highscore.json. Il est aussi assez simple d’améliorer graduellement le système : si on souhaite mettre en place un leaderboard, il suffit d’enregistrer une liste de JSON dans notre fichier et si on veut rajouter un nom d’utilisateur, on peut rajouter un champs au JSON. Dans le cas où on veut faire un leaderboard en ligne, on peut envoyer le JSON au serveur via HTTP.

Mise en place : Afin de pouvoir lire et écrire les scores, j’ai créé une classe ScoreJsonWriter. Afin d’éviter les conflits de lecture et d’écriture et afin de s’assurer de sa persistance entre scènes, la classe est un singleton qui ne se détruit pas lors des changements de scène. Un Singleton est un design pattern qui permet de s’assurer de l’unicité d’un objet. Plus de détails sur le design pattern singleton est fourni dans les parties suivantes.

Lors de son Enable, l'objet ScoreJsonWriter lit le high score écrit dans le fichier json et le stocke. A chaque fois qu'il reçoit un signal issu du channel HighScoreEventChannel, l'objet compare le score annoncé dans le Channel et si ce score est en effet un highscore, il met à jour la valeur en changeant la valeur enregistrée dans le JSON stocké localement. Les captures d'écran ci-dessous montrent comment les méthodes sont implémentées :

```
/*
A component to read and write High Score values to JSON.
*/

using System.IO;
using UnityEngine;

3 references
public class ScoreJsonWriter : ImmortalSingleton<ScoreJsonWriter>
{
    #region Event Channels
    [SerializeField]
    2 references
    private HighScoreEventChannelSO _highScoreEventChannelSO;
    #endregion

    #region Read/Write JSON HighScore
    2 references
    private string _jsonDirectory = "Assets/Data/highscore.json";

    4 references
    public class HighScoreJson
    {
        2 references
        public int HighScore;
    }
    #endregion

    5 references
    private int _highScore = 0;

    #region Getters and Setters
    1 reference
    public int GetHighScore()
    {
        return _highScore;
    }
    #endregion
}
```

Figure 56: Attributs de la classe ScoreJsonWriter. On possède un EventChannel pour réceptionner les nouveaux scores et toutes les informations pour lire le fichier de HighScore stocké localement.

```
#region MonoBehaviour methods
0 references
void OnEnable()
{
    ReadScoreFromJson();
    _highScoreEventChannelSO.onEventRaised += WriteScoreToJson;
}

0 references
void OnDisable()
{
    _highScoreEventChannelSO.onEventRaised -= WriteScoreToJson;
}
#endregion

#region Read and Write methods
1 reference
public void ReadScoreFromJson()
{
    string jsonString = File.ReadAllText(_jsonDirectory);
    HighScoreJson highScoreJson = JsonUtility.FromJson<HighScoreJson>(jsonString);
    _highScore = highScoreJson.HighScore;
}

/// <summary>
/// On signal, check if the declared score is a high score
/// Save it to JSON if it is the case.
/// </summary>
/// <param name="highScore">Declared score for the run</param>
2 references
public void WriteScoreToJson(int highScore)
{
    if (highScore > _highScore)
    {
        _highScore = highScore;
        HighScoreJson highScoreJson = new();
        highScoreJson.HighScore = _highScore;
        string jsonString = JsonUtility.ToJson(highScoreJson);
        File.WriteAllText(_jsonDirectory, jsonString);
    }
}
#endregion
```

Figure 57: Méthodes du ScoreJsonWriter. On lit la valeur du HighScore OnEnable et on vérifie si on bat effectivement le meilleur score à chaque fois qu'un event approprié est invoqué. On enregistre dans la foulée ce meilleur score afin d'éviter des pertes d'informations.

7 Design Pattern utilisés

Observer

Dans le cadre du jeu, le joueur est amené à pouvoir collecter des collectibles ainsi qu'à pouvoir toucher des obstacles, ce qui provoque des effets différents et multiples (quand le joueur rentre en collision avec un collectibles, il faut incrémenter le score, détruire le collectible ou encore mettre à jour la vitesse du jeu). Afin de pouvoir gérer ces différents effets, j'ai choisi d'utiliser le Design Pattern Observer : des objets s'abonnent à une Unity Action afin de pouvoir être notifié d'un événement et agir en conséquence. Afin de pouvoir programmer un composant réutilisable pour tous nos types d'événements, nous avons commencé par créer une classe parent appelée VoidEventChannel (image ci-dessous). Le rôle de cette classe est de nous donner un canevas pour programmer des UnityActions qui prennent en paramètre une variable.

```
/*  
A class to handle events with arguments.  
*/  
  
using UnityEngine;  
using UnityEngine.Events;  
  
[CreateAssetMenu(fileName = "VoidEventChannel_SO", menuName = "Events/VoidEventChannelSO")]  
18 references  
public class VoidEventChannelSO<T> : ScriptableObject  
{  
    36 references  
    public UnityAction<T> onEventRaised;  
  
    13 references  
    public virtual void RaiseEvent(T value)  
    {  
        onEventRaised?.Invoke(value);  
    }  
}
```

Figure 58: Le VoidEventChannel définit un canal pouvant passer toute sorte d'argument et permettant d'invoquer des Unity Actions.

```
/*  
Event to notify if the user's score should increase.  
*/  
  
using UnityEngine;  
  
[CreateAssetMenu(fileName = "ScoreEventChannel_S0", menuName = "Events/ScoreEventChannelS0")]  
0 references  
public class ScoreEventChannelS0 : VoidEventChannelS0<int> { }
```

Figure 59: Le ScoreEventChannel passe en argument la valeur en point associé au collectible récupéré. Cette valeur est un attribut du collectible et peut varier en fonction de sa nature.

Dans le cas des Collectibles, on possède par exemple un ScoreEventChannel qui prend comme paramètre un integer (image ci-dessous).

Le channel est ensuite utilisé dans l'objet de gestion des collisions qui lance un événement à chaque fois qu'une collision est faite. De son côté, l'objet de gestion des scores peut s'abonner à l'événement afin de pouvoir mettre à jour sa variable de score.

Les objets qui ont besoin d’être notifié lorsqu’un Collectibles est collecté peuvent alors déclarer une

```
/*
An abstract class in order to define states for our player.
*/

using System.Collections;

/// <summary>
/// Unity has its own StateMachineBehaviour states.
/// In our case, we did not need the extra complexity.
/// </summary>
10 references
public interface IState
{
    /// <summary>
    /// A method to enter a state (change graphics for the state, play state sound...)
    /// </summary>
    7 references
    public void Enter(PlayerStateMachine playerStateMachine, PlayerPowerUp playerPowerUp);
    6 references
    public void Update();
    7 references
    public IEnumerator Special();

    /// <summary>
    /// Method to Exit State
    /// </summary>
    7 references
    public void Exit();
}
```

Figure 61: Implémentation de notre interface pour les états. Chaque état spécifie ses transitions, son pouvoir spécial et sa mise à jour.

```
void OnDisable()
{
    collectableEventChannelSO.onEventRaised -= HandleCollectible;
    obstacleEventChannelSO.onEventRaised -= HandleObstacle;
}
#endregion

#region Handle collisions with items
2 references
private void HandleObstacle(Obstacle.Resistance resistance)
{
    _playerHealth.CollisionWithObstacle(resistance);
}

2 references
private void HandleCollectible(int score)
{
    _playerScore.IncrementScore(score);
}
#endregion
}
```

Figure 60: CollisionHandler : une classe pour gérer les collisions entre le joueur et les collectibles

Afin de pouvoir voir comment construire un état, voyons un exemple concret avec l'état `ChickenState`. Il s'agit du troisième état disponible pour le joueur. Lorsque le joueur atteint cet état, il débloquent l'aptitude spéciale qui lui permet de planer. Cela est représenté dans notre code par le fait que dans la fonction `Enter`, l'état `ChickenState` invoque la méthode pour débloquent l'aptitude correspondante. De façon similaire, les autres états débloquent des aptitudes lors des transitions. Les états gèrent aussi la logique associée à chacun des pouvoirs spéciaux. Dans l'exemple de code ci-dessous, on possède un pouvoir qui ralentit la vitesse du jeu pendant dix secondes. On lance des consignes pour lancer une animation particulière pour signaler au joueur que le pouvoir est en cours, on lance une méthode pour ralentir le niveau. On utilise une coroutine afin de pouvoir garantir que le pouvoir spécial dure la durée spécifiée et ensuite on rétablit la situation initiale (animation normale et vitesse normale de jeu). Ce design pattern m'a permis de pouvoir commencer à tester les changements d'état dès que j'avais deux états disponibles et j'ai pu ainsi les rajouter au fur et à mesure que je voyais comment les implémenter. Tous les pouvoirs spéciaux n'ayant pas été mis en place, je sais que je peux modifier chacun des états individuellement sans impacter le code des autres états.

Remarque : Normalement, les états devraient prendre plus de responsabilités. Notamment, au moment où l'on `Enter` un état, il faudrait indiquer au joueur quel modèle 3D utiliser pour représenter son personnage (on utiliserait alors une classe pour gérer le chargement et la transition des modèles 3D). Au moment où le joueur quitterait un état (dans `Exit`), on utiliserait aussi une méthode afin de pouvoir ajouter un effet lumineux pour indiquer la transition et cacher au joueur la transition entre les deux modèles 3D. Malheureusement, du fait de la période assez courte de développement, je n'ai pas eu le loisir d'implémenter les méthodes décrites ci-dessus.

```
/*  
The ChickenState. it is the third state the player is in.  
*/  
  
using System.Collections;  
using UnityEngine;  
  
1 reference  
public class ChickenState : IState  
{  
    5 references  
    private PlayerStateMachine _playerStateMachine;  
    1 reference  
    private int _powerUpDuration = 10;  
  
    2 references  
    public void Enter(PlayerStateMachine playerStateMachine, PlayerPowerUp playerPowerUp)  
    {  
        playerPowerUp.EnablePower(PlayerPowerUp.PlayerPowerEnum.Glide);  
        _playerStateMachine = playerStateMachine;  
    }  
  
    1 reference  
    public void Update() { }  
  
    /// <summary>  
    /// Implement special effect here.  
    /// Real effect not implemented here. We use the Chick's effect right now.  
    /// </summary>  
    /// <returns></returns>  
    2 references  
    public IEnumerator Special()  
    {  
        _playerStateMachine.SpecialAnimation(true);  
        _playerStateMachine.InvokeChangeLevelSpeed(.5f);  
        yield return new WaitForSeconds(_powerUpDuration);  
        _playerStateMachine.InvokeChangeLevelSpeed(2f);  
        _playerStateMachine.SpecialAnimation(false);  
    }  
  
    2 references  
    public void Exit() { }  
}
```

Figure 62: Implémentation concrète d'un état. Notre état possède des aptitudes qui se débloquent lors de sa création et un pouvoir spécial.

Le dernier composant de notre StateMachine est la StateMachine qui prend en charge la transition avec les états. Dans mon cas, la state machine gère aussi la gestion du nombre de charge de pouvoir spécial disponible pour le joueur. Normalement il aurait mieux valu séparer la gestion des états

et celle des charges de pouvoir spécial afin de respecter le principe de Single Responsibility. Etant donné la faible taille du projet et le fait qu'à terme les pouvoirs spéciaux ne se déclencheraient pas sur demande mais en touchant un collectible particulier (évoqué dans la partie sur le scope final du projet), j'ai laissé les choses telles quel.

Singleton

Dans le code, je disposais de plusieurs éléments dont je désirais m'assurer l'unicité. Afin de pouvoir garantir cette condition, j'ai utilisé le design pattern Singleton. Le but de ce design pattern est de garantir qu'on ne possède qu'un seul exemplaire d'un service en vérifiant si à la création on ne possède pas une instance qui existe déjà. Si c'est le cas, on retourne l'instance existante plutôt qu'une nouvelle instance. L'implémentation du design pattern est présenté dans les deux capture d'écran en dessous. Afin de garantir l'unicité du singleton, on lui associe une variable publique et statique appelée Instance. Quand l'Instance du Singleton est appelée, on cherche s'il existe une instance du singleton et si c'est le cas, on la retourne. S'il n'existe pas d'instance du singleton, on en crée une et on supprime les doublons en utilisant la méthode RemoveDuplicates disponible aussi en dessous.

```
1  /*
2  A class to create singletons and make sure that any duplicate instance gets deleted.
3  There must be only one singleton per scene.
4  */
5
6  using UnityEngine;
7
8  10 references
9  public class Singleton<T> : MonoBehaviour
10     where T : Component
11 {
12     #region Singleton access
13     10 references
14     private static T _instance;
15     5 references
16     public static T Instance
17     {
18         get
19         {
20             if (_instance == null)
21             {
22                 _instance = (T)FindFirstObjectByType(typeof(T));
23                 if (_instance == null)
24                 {
25                     SetupInstance();
26                 }
27             }
28             return _instance;
29         }
30     }
31     #endregion
32
33     #region MonoBehaviour methods
34     8 references
35     public virtual void Awake()
36     {
37         RemoveDuplicates();
38     }
39
40     1 reference
41     private static void SetupInstance()
42     {
43         _instance = (T)FindFirstObjectByType(typeof(T));
44         if (_instance == null)
45         {
46             GameObject gameObj = new GameObject();
47             gameObj.name = typeof(T).Name;
48             _instance = gameObj.AddComponent<T>();
49             //DontDestroyOnLoad(gameObj);
50         }
51     }
52     #endregion
```

Figure 63: Design pattern Singleton : attributs et création.

```
1 reference
private void RemoveDuplicates()
{
    if (_instance == null)
    {
        _instance = this as T;
        //DontDestroyOnLoad(gameObject);
    }
    else if (_instance != this)
    {
        Destroy(gameObject);
    }
}
```

Figure 64: La méthode *RemoveDuplicates* vérifie si l'instance créée à l'instant est l'Instance du singleton. Si ce n'est pas le cas, cela veut dire qu'il existe une autre instance et donc qu'on n'a pas besoin d'en créer une nouvelle.

Le design pattern Singleton est par exemple utilisé pour le *PlayerController* (la classe qui gère la lecture des actions du joueur).

```
[RequireComponent(typeof(Collider))]
1 reference
public class PlayerController : Singleton<PlayerController>
```

Figure 65: La classe gérant les Actions du joueur est un Singleton.

Dans mon cas, je possède des menus et un niveau d'infinite runner. Je n'avais pas besoin que mes singletons persiste d'une scène à l'autre et c'est pour cela que je n'ai pas utilisé la méthode *DontDestroyOnLoad*. Par contre, pour le gestionnaire de son, j'ai besoin que le composant soit unique et persiste d'une scène à l'autre pour que la musique ne redémarre pas lors d'une transition d'un niveau à l'autre. Afin de pouvoir m'adapter à ce besoin, j'ai créé une classe enfant appelée *ImmortalSingleton*

dont le but est de reprendre la logique du Singleton en rajoutant comme contrainte le fait que l'ImmortalSingleton doit persister entre les niveaux. Cela se fait en rajoutant au moment de la création du Singleton la méthode DontDestroyOnLoad.

```
/*  
A special class of singletons for singletons which persist between scenes.  
*/  
  
using UnityEngine;  
  
1 reference  
public class ImmortalSingleton<T> : Singleton<T>  
|   where T : Component  
{  
    6 references  
    public override void Awake()  
    {  
        base.Awake();  
        if (Singleton<T>.Instance == this)  
        {  
            DontDestroyOnLoad(this.gameObject);  
        }  
    }  
}
```

Figure 66: L'ImmortalSingleton persiste entre les scènes.

8 Pratiques pour faciliter la lisibilité du code

Utilisation de Github : Afin d'éviter de perdre du travail effectué ou encore de pouvoir échanger plus facilement sur le code de façon asynchrone avec les autres développeurs dans le groupe, j'ai décidé de mettre le code de mon jeu sur Github. Cela m'a aussi encouragé à expérimenter et à faire des tests car je savais que mon code était de toute façon archivée.

Demande d'avis extérieurs : Ne sachant pas trop si mon code était compréhensible, j'ai demandé en fin de projet à deux de mes amis extérieurs au cours Gaming Campus de me donner leur retour sur le code. Ils ont eu la gentillesse de m'indiquer ce qu'ils ne comprenaient pas et ce qu'ils trouvaient flou. Grâce à leur retour, j'ai essayé de refactorer les composants les plus confus (j'ai notamment séparé plusieurs des scripts liés au joueur qui étaient auparavant liés ensemble). Leur point de vue extérieur m'a permis de comprendre comment essayer de mettre en valeur les scripts les plus importants et d'adapter le nommage de plusieurs de mes classes.

Utilisation d’un linter : Afin de pouvoir faciliter la mise en forme du code et uniformiser la manière dont le code est écrit, j’ai choisi d’utiliser un linter appelé c sharpier (<https://csharpier.com/>). Le linter permet notamment d’insérer des retours à la ligne entre les fonctions ou des les cas où l’on possède des lignes trop longues ou encore de supprimer les espaces en trop en fin de lignes ou de scripts. Le linter est invoqué de manière automatique lors d’un commit du code grâce à un pre-commit hook. Code du pre-commit hook disponible dans le dossier “Scripts>pre-commit.sh” ou encore ci-dessous :

```
Scripts > $ pre-commit.sh
1  #!/bin/sh
2  echo "Pre-commit hook is running"
3  dotnet tool run csharpier format .
4  if [ $? -ne 0 ]; then
5      echo "CSharpier formatting failed. Please fix formatting before committing."
6      exit 1
7  fi
```

Figure 67: Pre-commit hook utilisé afin de lancer automatiquement mon linter pour réorganiser mon code et le rendre plus propre.

Utilisation de régions : Certains scripts possèdent beaucoup de code et il peut être compliqué de les naviguer. Afin de faciliter cela, vs code possède la possibilité de définir dans le code des “region” par le biais de commentaires (#region pour créer une région et #endregion pour la fermer). Leur but est de regrouper les éléments de code de même nature. Un exemple de région est précisé dans la capture d’écran ci-dessous.

On y voit un exemple de région pour stocker l’ensemble des variables liées aux actions du joueur. Les régions apparaissent à droite de l’écran dans le navigateur afin de savoir où on se trouve dans le code et où se trouvent chaque région.

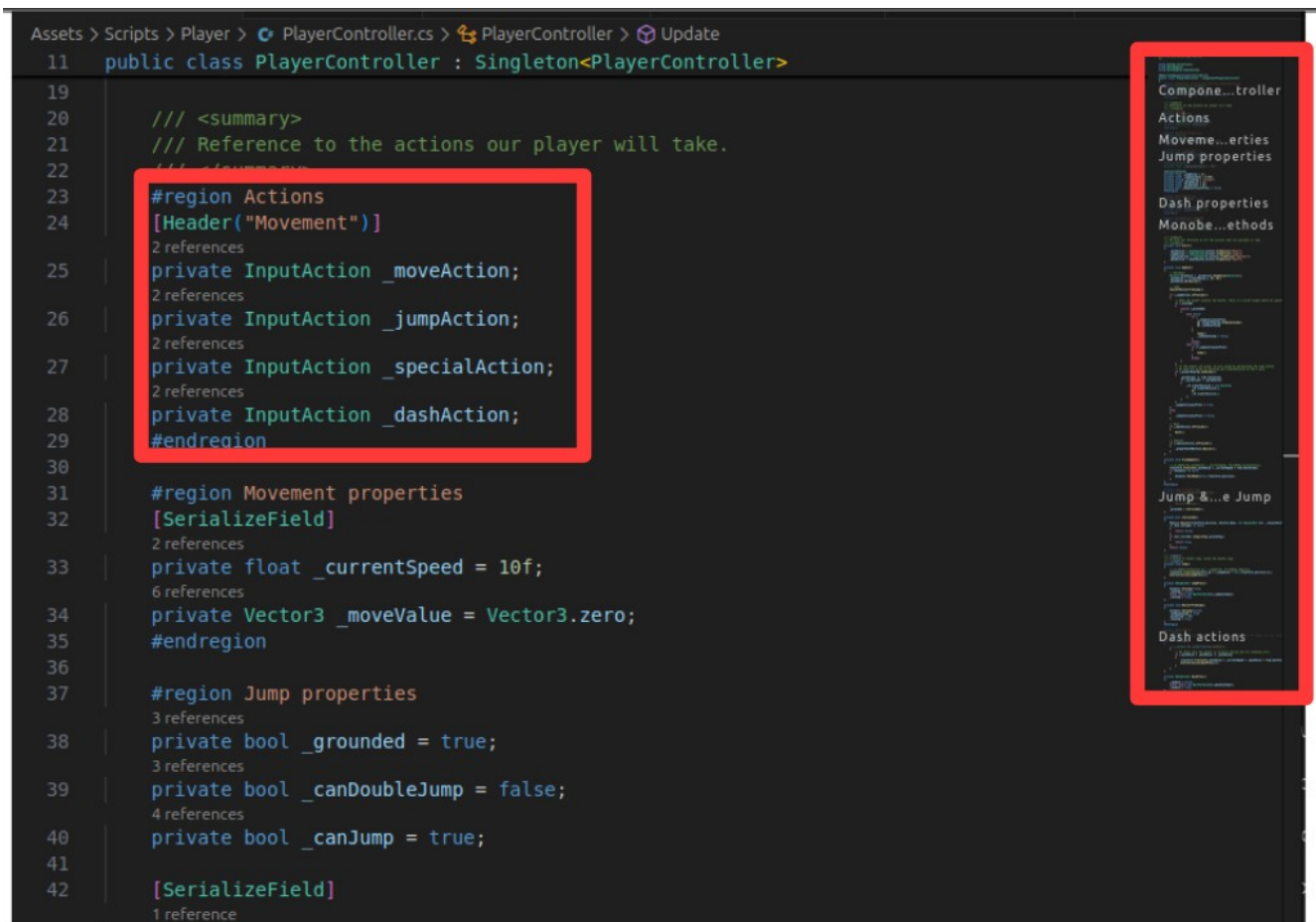


Figure 68: Les régions permettent de localiser où se trouvent les composants du code. Elles forcent aussi à s'adapter en regroupant les éléments similaires dans les régions associées.

D Conclusion

Ce projet a été pour moi l'occasion de me mettre au défi et d'essayer de développer mes compétences de programmation dans un cadre bienveillant. Je suis très heureux d'avoir pu échanger avec des gens passionnés en informatique et en programmation jeu vidéo.

Informatique

Travailler avec Unity a été un vrai challenge. Il a fallu changer ma façon de penser et intégrer une logique différente et que je ne connaissais pas. Avec du recul, je me rends compte que je n'ai pas tiré partie des avantages qu'offrent Unity. En effet, je possède encore beaucoup d'éléments éparpillés et suivre les évolutions de chacun d'entre eux n'a pas été facile au cours du projet. J'aurais dû essayer de plus tirer partie de l'interface et limiter le nombre d'éléments qui ont lieu de façon « cachée » (comme les Events. Si je dois travailler de nouveau avec un nouvel outil, j'essaierai de passer plus de temps à essayer de comprendre ses spécificités avant de me lancer dans la programmation.

Le fait de pouvoir travailler de façon étendue sur un projet m'a forcé à revoir mon code et à constamment essayer de le simplifier. Le fait de refactoriser mon code n'est pas quelque chose que je fais beaucoup sur mes projets personnels. Être obligé d'expliquer ton code à quelqu'un d'autre m'a forcé à soigner la lisibilité.

Le fait d'avoir un projet vaste avec beaucoup de fonctionnalités oblige à prioriser le travail. Lors des premières semaines de travail, prioriser les tâches et comprendre ce que j'étais censé faire était un défi. Je suis heureux d'avoir pu faire l'exercice. **La prochaine fois, j'essaierai de moins m'inquiéter sur les fonctionnalités pour passer plus de temps au début pour m'habituer à l'outil.** Me donner une semaine pour me familiariser avec l'outil m'aurait sans doute aidé à savoir où aller.

Travail en équipe

Avec du recul, je pense que **j'aurais dû communiquer davantage tout au long du projet.** Le fait que beaucoup de temps ait été passé avec des résultats faibles (les quatre premières semaines) a été déprimant et a sans doute compliqué la tâche des game designers qui ont davantage été sollicités en fin de projet.

Le fait d'**avoir des jalons intermédiaires dans le projet a été une grande aide** car cela me permettait d'évaluer le travail à accomplir plus précisément. Cela faisait aussi que je pouvais designer mes composants avec le reste du projet en tête.

Perspective

S'il est encore flou pour moi de savoir quelle est la prochaine étape, je sais que **je suis en train d'apprendre des compétences et qu'il va falloir que je les consolide si je veux progresser**. Le fait de travailler avec Unity m'a permis de tester directement la construction de niveau et l'élaboration de la logique de jeu. Cela m'a aussi rendu d'autant plus admiratif pour les constructeurs de moteurs de jeu. Je pense que la prochaine étape pour moi serait d'essayer de participer à des Game Jam afin de prendre le temps de faire mes premiers jeux. Cela me permettra aussi d'avoir un environnement de travail qui m'encourage à faire les choses bien et avec des deadlines.

Merci d'avoir lu, passez une excellente journée.

E Annexes

1 Procédure de test

Note sur le logging : Afin d'éviter que le code soit trop chargé, les commandes pour écrire des logs ont été supprimés du code avant de le poster sur Github et ont été enlevés du code livré dans le dossier en complément de ce dossier.

Test des sauts

Un des éléments qui a fait l'objet de nombreuses itérations est le saut. Comme la sphère utilise un rigidbody, j'avais pensé utiliser les méthodes du rigidbody pour mon saut avec AddForce. Le problème est que cela donnait un saut lent, ce qui ne fonctionnait pas pour un InfiniteRunner qui demande des réflexes : quand il appuie sur le saut, le joueur doit être dans les airs. C'est pour cela que j'ai changé le saut en un transform.Translate pour traduire le joueur. Le mouvement brusque est adouci par le fait que le modèle 3D se déplace plus lentement pour rejoindre le collider qui est déjà dans les airs.

A screenshot of a code editor showing a C# script for a jump function. The code is enclosed in a region marked with #region and #endregion. It includes comments and logic for moving the player's position and updating the animator.

```
0 references | Unity Message
private void FixedUpdate()
{
    // _rb.AddForce(_moveValue * _currentSpeed, ForceMode.Acceleration);
    transform.Translate(_moveValue * _currentSpeed * Time.deltaTime);
    if (Animator != null)
    {
        Animator.MoveModel(this.transform.position);
    }
}
#endregion
```

Figure 69: Des tests successifs ont permis de calibrer la hauteur du saut et son mode de fonctionnement.

Test des événements

Une des difficultés à gérer est le fait que beaucoup des opérations dans le code ont lieu grâce à des Events, de façon « cachée ». En effet, quand un événement se déclenche, il faut savoir quelles sont les réactions escomptées. La première étape pour pouvoir déboguer ces événements a été de faire une table qui indique pour chacun des EventChannels quels sont tous les autres éléments qui utilisent l'EventChannel. On ne compte pas dans le tableau les ScriptableObject qui ne sont là que pour changer les caractéristiques des objets (Par Exemple ChunkCharacteristicsSO donne des informations sur un

chunk de niveau mais ne déclenche aucun événement). On obtient ainsi le tableau suivant (classement par ordre alphabétique) :

Nom de l'Event Channel	Classes pouvant lever l'éven	Classe pouvant réceptionner l'évent
VoidEventChannelSO	N/A	
ChangeLevelSpeedSO	PlayerStateMachine	ChunkMover
DashEventChannelSO	Input	PlayerController
HighScoreEventChannelSO	PlayerScore	ScoreJsonWriter
JumpEventChannelSO	Input	PlayerController
MoveEventChannelSO	Input	PlayerController
NoHealthEventChannelSO	PlayerHealth	GameOverHandler PlayerScore
ObstacleCollisionEventChannelSO	Obstacle	CollisionHandler
ScoreEventChannelSO	Corn SuperCorn SpecialCorn	AudioManager CollisionHandler
SetHealthEventChannelSO	PlayerStateMachine	PlayerHealth
SpecialAnimationChannelSO	PlayerStateMachine	PlayerAnimator
SpecialEventChannelSO	Input	AudioManager PlayerController
SpecialReloadEventChannelSO	SpecialCorn	PlayerStateMachine
StateChangeEventChannelSO	PlayerScore	ChunkMover PlayerPowerUp PlayerStateMachine

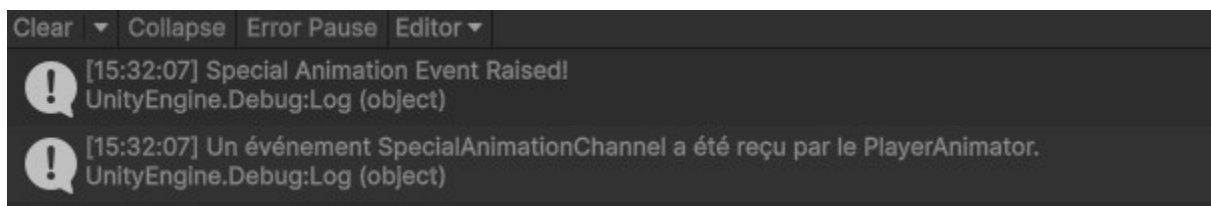
Le tableau était enrichi à chaque fois qu'un nouvel event était crée. Avant de pouvoir tester les effets de mon EventChannel, je commençais par vérifier que l'événement était bien levé au bon moment et que tous les objets concernés recevaient bien le signal. Pour cela, je profitais du fait que tous les EventChannels héritaient de la classe VoidEventChannelSO qui possède une méthode virtuelle RaiseEvent. Pendant la phase de test, je surchargeais la méthode afin de pouvoir écrire dans les logs qu'un événement avait bien été levé et je vérifiais de même que chacune des classes pouvant réceptionné l'événement le recevait bien grâce aux logs.

```
Event to notify the Animator that he should change the player Animation.  
*/  
  
using UnityEngine;  
  
[CreateAssetMenu(  
    fileName = "SpecialAnimationChannel_SO",  
    menuName = "Events/SpecialAnimationChannelSO"  
)]  
2 references  
public class SpecialAnimationChannelSO : VoidEventChannelSO<bool>  
{  
    13 references  
    public override void RaiseEvent(bool value)  
    {  
        Debug.Log("Special Animation Event Raised!");  
        onEventRaised?.Invoke(value);  
    }  
}
```

Figure 70: Exemple avec le SpecialAnimationChannelSO : une trace est ajoutée pour identifier quand est-ce que l'événement est provoqué.

```
public void SetSpecial(bool specialValue)  
{  
    Debug.Log("Un événement SpecialAnimationChannel a été reçu par le PlayerAnimator.");  
    _playerAnimator.SetBool("Special", specialValue);  
}
```

Figure 71: Du côté du récepteur, une trace est ajoutée. Quand on possède plusieurs récepteurs, les messages de trace sont différenciés pour pouvoir vérifier quel composant a reçu l'événement.



```
Clear ▼ Collapse Error Pause Editor ▼  
[15:32:07] Special Animation Event Raised!  
UnityEngine.Debug:Log (object)  
[15:32:07] Un événement SpecialAnimationChannel a été reçu par le PlayerAnimator.  
UnityEngine.Debug:Log (object)
```

Figure 72: On peut ainsi vérifier que tous les composants appropriés reçoivent les Events correspondants.

Chacun des EventsChannels sont testés séparément afin de s'assurer de leur fonctionnement individuel et éviter d'avoir des logs surchargés.

Test à réaliser lors de l'introduction d'un nouveau EventChannel : Ajouter un Debug.Log lorsque l'événement se produit et faire en sorte que tous les récepteurs aussi affichent un Debug.Log afin de s'assurer que les Events ont bien été réceptionnés.

Test des aptitudes

Les aptitudes se débloquent au fur et à mesure du jeu. Vérifier si les aptitudes sont débloquentes ou pas se fait via une classe PlayerPowerUp via des variables privées. Afin de pouvoir accélérer les tests et ne pas avoir à attendre de pouvoir débloquent les pouvoirs spéciaux, toutes les variables gérant les aptitudes spéciales sont des variables [SerializeField] afin de pouvoir les cocher et décocher et ainsi vérifier comment les aptitudes se comportent en jeu.

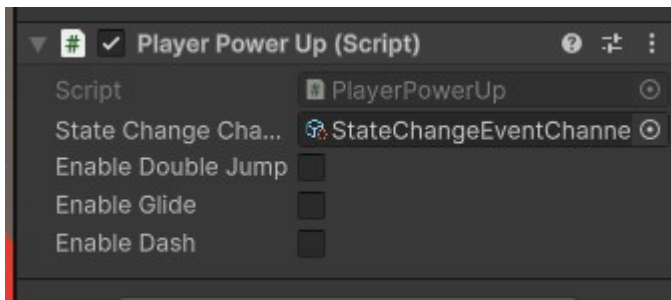


Figure 73: Il est possible de préciser quelles aptitudes débloquent dès le début de la run afin de pouvoir tester les aptitudes sans avoir à attendre qu'elles se débloquent « naturellement ».

Test à réaliser : Afin de tester toutes les aptitudes, **il faut essayer de tous les décocher puis les cocher les uns après les autres afin de pouvoir les tester individuellement puis en combinaisons.** Notamment : le Glide et le double saut doivent être testés ensemble car ils influent l'un sur l'autre.

Test des Chunks de niveaux

Une classe Spawner gère quels sont les chunks de niveau qui peuvent apparaître en jeu. Pour faire cela, il regarde quels sont les chunks disponibles dans un dossier de prefabs et vérifie pour tous les chunks quelles sont leur probabilité d'apparition. Cette probabilité est contrôlée par un ScriptableObject et il est donc facile de la changer pour « encourager » les chunks que l'on souhaite tester. Chaque fois que je créais un nouveau chunk, j'augmentais sa probabilité d'apparition. Je conservais une probabilité supérieure à zéro pour les autres chunks car il est aussi important de pouvoir contrôler comment les chunks de niveau s'emboîtent les uns aux autres.

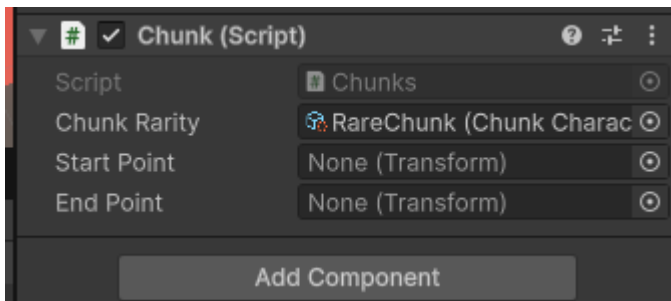


Figure 74: Il est possible de pouvoir contrôler la fréquence d'apparition de chacun des chunks de niveau.

Afin de pouvoir garantir que les situations étaient reproductibles, pendant toute la durée des tests, j'ai fixé la seeds pour que le hasard se comporte de la même manière.

```
1 reference  
private int __randomSeed = 42;
```

Figure 75: La Seed de hasard est fixé pendant la durée des tests.

Test à réaliser lors de l'introduction d'un nouveau chunk : Afin de tester la génération de certains chunks en particulier, il faut mettre les chunks concernés dans le dossier « ChunkTests » et modifier dans le code du Spawner le nom du dossier dans lequel prendre les Chunks à « ChunkTests ». Il faut aussi changer la rareté du chunk à TestChunk (Captures d'écran ci-dessous).

```
2 references  
private string __chunkFolderName = "Chunks";  
1 reference
```

Figure 76: Pour tester un chunk en particulier, il faut le mettre dans le dossier ChunkTests et changer le nom du dossier de chunks à ChunkTests.

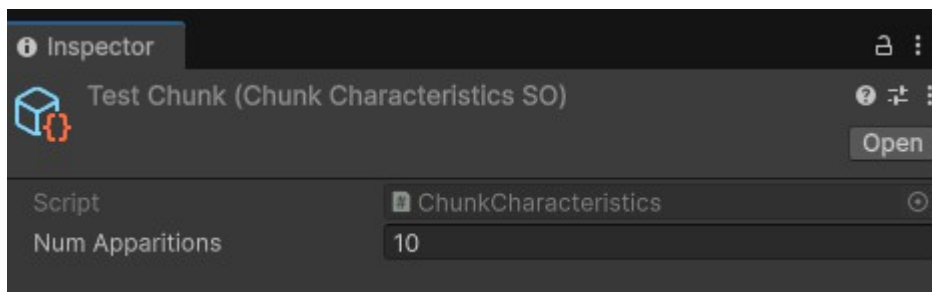


Figure 77: La rareté "TestChunk" permet de faire en sorte qu'un chunks soit très commun. Idéal quand on veut le tester.

Tests papiers

Lorsque j'ai réalisé qu'il fallait que je crée des chunks de niveau afin de pouvoir mettre en place les mécaniques d'infinite runner, j'ai commencé par faire des dessins pour pouvoir me mettre d'accord sur ce qui devait constituer un chunk et comment le joueur était sensé le parcourir. Cela a mené à plusieurs dessins au brouillon qui m'ont permis de stabiliser mes idées et ce que je souhaitais dessiner comme tronçons de niveau.

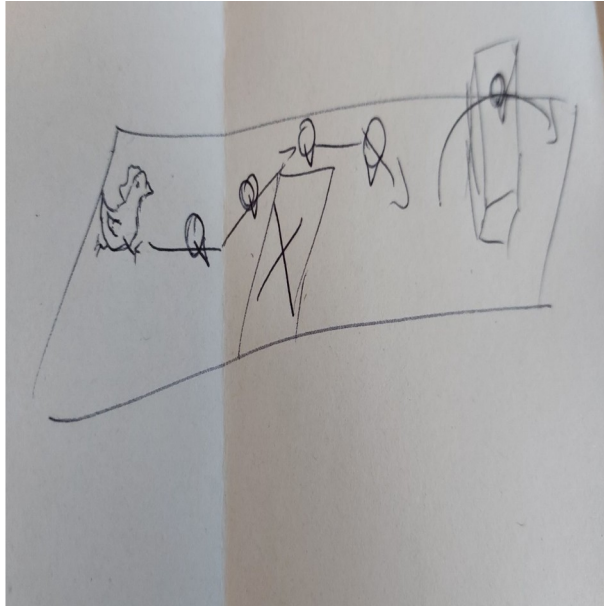
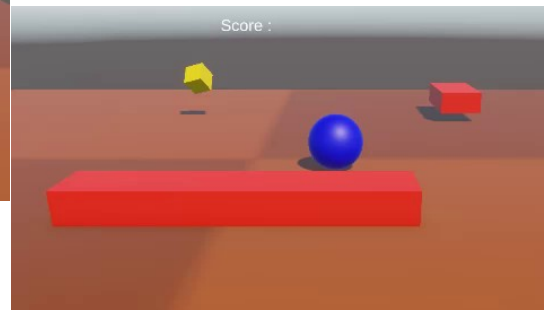
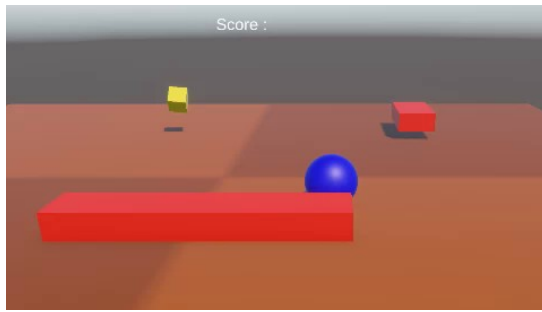


Figure 78: Exemple de croquis dessiné lors de l'étape de création de chunks. Les grains sont dessinés afin de guider le joueur.

Scène de test

Le jeu possède de nombreux paramètres à régler (hauteur de saut, temps de cooldown entre deux sauts, architecture des chunks de niveaux...). Tout ces paramètres doivent être calibrés et il faut pour cela faire des tests. Je voulais me donner un bac à sable dans lequel je pouvais modifier chacun des éléments séparément sans impacter le reste de mon jeu. Pour cela, j'ai créé une scène bac à sable dans laquelle je pouvais déplacer mon personnage sans contrainte. Dans cette scène étaient entreposés tous mes assets afin de pouvoir voir comment mon joueur se comportait par rapport à eux. Ci-dessous se trouvent deux vidéos qui montrent comment au début du jeu j'utilisais l'environnement afin de pouvoir faire des essais avec différentes valeurs de variables :



Drawing 2: Ces vidéos présentent quelques essais avec des paramètres différents et des configurations de caméras différentes afin de s'assurer que le déplacement du joueur soit agréable et adapté au jeu.

On trouve ci-dessous un exemple plus avancé où je testais mes premiers chunk de niveau afin de m'assurer que je pouvais les traverser et qu'ils étaient suffisamment lisibles en jeu.

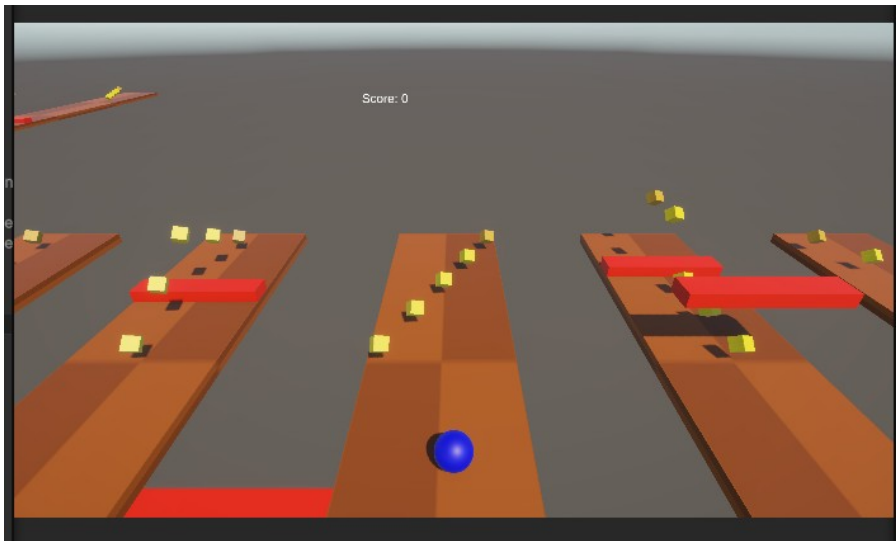


Figure 79: Scène de test lors d'une phase plus avancée de développement. Je pouvais tester chacun des assets séparément sans impact sur mon jeu.

J'ai réalisé plus tard dans le projet que j'aurais dû faire un build de cette scène de test pour laisser les game designers expérimenter et ainsi me donner des feedbacks tout au long du projet. C'est une occasion manquée.

PlayTest

Afin de pouvoir m’assurer d’avoir un avis supplémentaire sur le jeu et vérifier que je ne possédais pas d’erreur j’ai demandé à un de mes amis s’il était possible qu’il teste le jeu afin de trouver s’il repérait des erreurs. La séance de playtest a permis d’identifier un bug : lorsque le joueur quitte la zone de jeu et tombe de la plateforme, il est censé faire une chute mortelle et le jeu s’arrête. Cette fin de jeu se déclenchait par le fait que j’ai placé un obstacle gigantesque en dessous de la zone de jeu. Le joueur en tombant touche l’obstacle et perd sa vie, ce qui provoque un Game Over. **Quand le joueur utilisait un de ses pouvoirs, il pouvait survivre à l’impact et continuait de chuter indéfiniment. Afin de résoudre ce bug, j’ai placé un deuxième obstacle en dessous du premier pour m’assurer que le joueur perde bien une deuxième vie au cas où.**

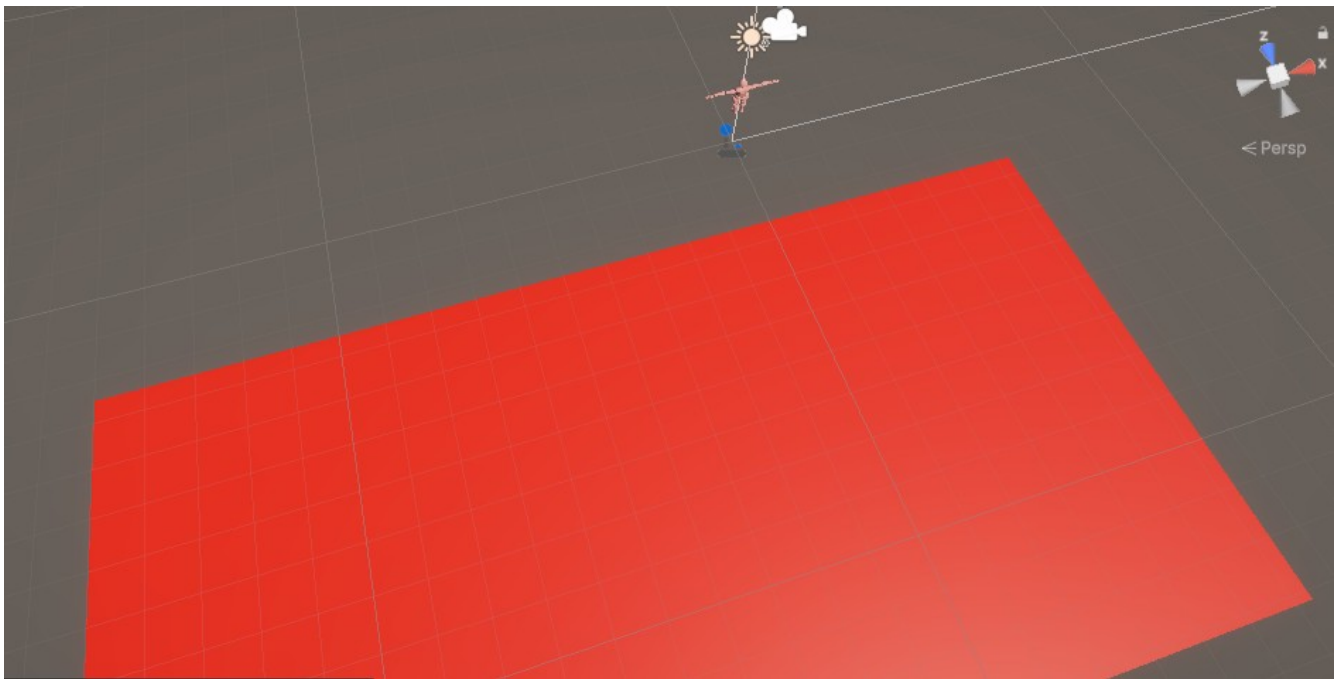


Figure 80: En dessous de la zone de jeu se trouve un grand obstacle pour que le joueur meure bien s'il tombe de la zone de jeu.

Autres bugs repérés et corrigés

Avant de faire de l'object pooling pour le spawner, je laissais le spawner créer un nouveau chunk de niveau à chaque fois que nécessaire ce qui faisait que le CPU était constamment sollicité et en utilisant le profiler, j'observais des pics d'activité réguliers. Afin de régler cela, le Spawner génère tous les chunks nécessaires en lors de la transition vers un nouveau niveaux. Les chunks sont ensuite activés et désactivés mais toujours recyclés, ce qui fait que l'activité du cpu est constante lors du niveau, comme l'indique le graphe ci-dessous. Les seuls pics interviennent lors des transitions entre niveaux (et on peut sans doute les dissimuler grâce à des écrans de chargement).

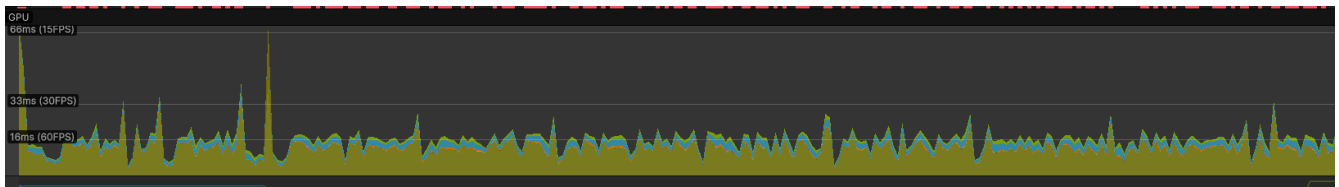


Figure 81: Le Spawner crée tous les chunks de niveau en début de niveau, ce qui fait qu'on possède un pic d'activité lors d'une transition de scènes (où on passe à 15 FPS) mais ensuite, l'activité est constante et on parvient à maintenir 30 FPS constamment.

2 Suivi des échanges lors du projet

Réunions d'équipes

[TEMPLATE]

Réunion #1

Participants:

-

Points abordés :

-

Actions réalisées :

-

Actions à mener pour la prochaine session :

-

Répartition du travail entre chaque membre du groupe pour la prochaine session :

-

Date de la prochaine session :

Transcript de la réunion :

Réunions

Réunion #1 - 11/07/2025

Participants:

- Antonio CARDOSO
- Frédéric LAURENZI
- Juan OSPINA
- Clément LION
- Baptiste LEVESQUE
- Andrea PALUMBO
- Antoine DAVID

Points abordés :

- Première mise en relation entre certains membres du groupe
- Mise au point sur les différentes deadlines

Actions réalisées :

- Discussions
- Premier brainstorming sur Miro

<https://miro.com/welcomeonboard/>

Q1hRRDdkV1V4Y1Y4YVIPcGYvcWd1Q3hsSE15QkJ3eGJSMWxUZjQ3M0ZLTVpDK0ZHTGdORmIVbU9vM2Z3U294SFpMMU9GbnV5aIVjNCtBbys1S0sySHNiYUdjaVMzNzJmeWVSQm52UWpZdXlvVzRrREE5b0FHanZYVnFPQ25rUXpyVmtkMG5hNDA3dVlncnBvRVB2ZXBnPT0hdjE=?share_link_id=283277977527

Actions à mener pour la prochaine session :

- Ajouter des idées sur le Miro / approfondir

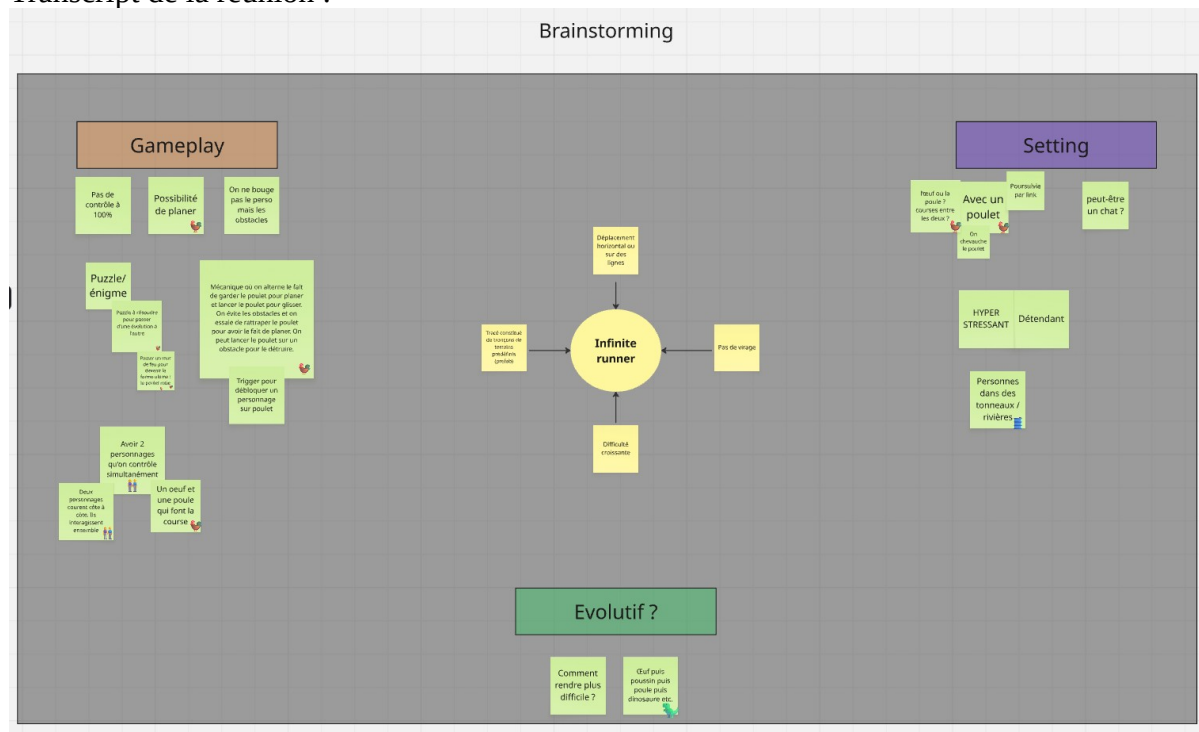
Répartition du travail entre chaque membre du groupe pour la prochaine session :

- Tout le monde (spécifique Game Designers)

Date de la prochaine session :

A déterminer.

Transcript de la réunion :



Réunion #2 - 21/07/2025

Participants:

- Andrea PALUMBO
- Antonio CARDOSO
- Baptiste LEVESQUE
- Benoit GUILLOUF
- Frédéric LAURENZI
- Hernani DIREITO
- Clément LION
- Vanina MEGANCK

Points abordés :

- Récap de la précédente session pour ceux qui n’y avait pas assisté

Actions réalisées :

- Proposition d’un autre type de jeu (Métro runner - cf Miro) + Brainstorming dédié
- Ajout d’idée sur les gameplay de la session précédente
- Ajout de nouveaux thèmes

Actions à mener pour la prochaine session :

- Commencer à réfléchir à une ébauche de piliers : à la fin de la prochaine session, les piliers doivent être choisis
- Ajouter des idées en plus sur les brainstorming

Répartition du travail entre chaque membre du groupe pour la prochaine session :

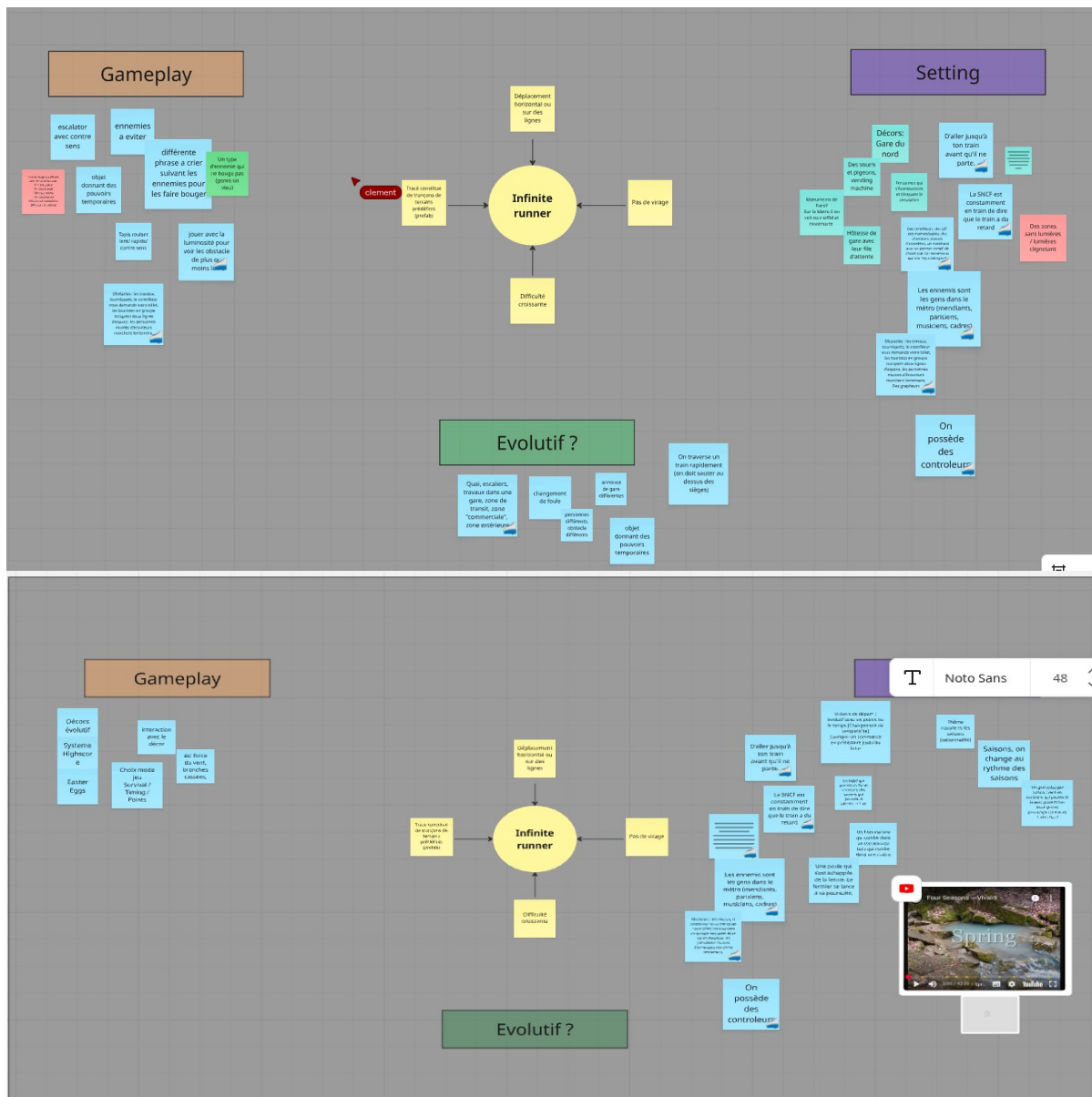
- Tout le monde (spécifique Game Designers)

<https://miro.com/welcomeonboard/>

Q1hRRDdkV1V4Y1Y4YVIPcGYvcWd1Q3hsSE15QkJ3eGJSMWxUZjQ3M0ZLTVpDK0ZHTGdORmlVbU9vM2Z3U294SFpMMU9GbnV5aIVjNCtBbys1S0sySHNiYUdjaVMzNzJmeWVSQm52UWpZdXlvVzRrREE5b0FHanZYVnFPQ25rUXpyVmtkMG5hNDA3dVlncnBvRVB2ZXBNPT0hdjE=?share_link_id=283277977527

Date de la prochaine session : À définir (sondage sur discord)

Transcript de la réunion :



Réunion #3

Participants:

- Clément LION
- Hernani DIREITO
- Antonio CARDOSO
- Antoine DAVID
- Julien SEQUERT

Points abordés :

- Choisir les piliers

- Le thème du poulet s'enfuyant du poulailler
- Evolution: oeuf, poussin, poulet, poulet rôti, super poulet, dinosaure, poulet ovni, redevenir un oeuf ? (changement de zone)

Actions réalisées :

- organisation entre les devs et GD

Actions à mener pour la prochaine session :

- D'ici deux semaines, les GD doivent réfléchir plus concrètement au projet, apporter des idées de mécaniques et éclairer le sujet pour en débattre ensemble et commencer le GDD
- **se voir entre GD pour uniformiser nos idées et avoir un vrai projet à évoquer aux devs.**
- identifier la boucle de Gameplay Macro

Répartition du travail entre chaque membre du groupe pour la prochaine session :

- Réunion à organiser entre GD

Date de la prochaine session :

- A définir (sondage Discord)

Réunion #4

Participants:

- Andrea PALUMBO
- Antonio CARDOSO
- Baptiste LEVESQUE
- Benoit GUILLOUF
- Antoine DAVID

Points abordés :

- Récap de la réunion précédente
- Le Game concept, ainsi que définition des grands principes du jeu
- Avancée dans les premières mécaniques

Actions réalisées :

- **Entamer la rédaction du GDD** : Overview, Coeur du jeu, Les 3C

Actions à mener pour la prochaine session :

- Transmettre tout cela aux développeurs et en débattre avec eux.

Répartition du travail entre chaque membre du groupe pour la prochaine session :

- Compléter les différentes parties du **GDD** pour reconfirmer avec l'équipe GD lors d'une prochaine réunion.

Date de la prochaine session :

- **Vendredi 21 Août 2025** (18h)

Transcript de la réunion :



<https://docs.google.com/document/d/1yzGHrd9vNmSgWG10nRx4GZU-BGr06pvyhv76LC-BoQ4/edit?usp=sharing>

Réunion #5

Participants:

- Baptiste Levesque
- Antonio Cardoso
- Frédéric Laurenzi
- Hernani Direito

Points abordés :

- Présentation de l'ébauche du GDD
- Explication du fonctionnement du doc et des rôles

Actions réalisées :

- Présentation des piliers, des premières mécaniques, boucles de gameplay, contrôles, etc ... aux développeurs
- Discussion sur la base du GDD

Actions à mener pour la prochaine session :

- Développeurs : Faire une réunion entre dev pour discuter du GDD
- Game designer : Continuer à étoffer le GDD

Répartition du travail entre chaque membre du groupe pour la prochaine session :

- Développeurs : Faire une réunion entre dev pour discuter du GDD
- Game designer : Continuer à étoffer le GDD

Date de la prochaine session : À définir pour une réunion entre dev et une réunion entre Game designer séparément

Transcript de la réunion :

<https://docs.google.com/document/d/1yzGHrd9vNmSgWG10nRx4GZU-BGr06pvyhv76LC-BoQ4/edit?usp=sharing>

Réunion #6

Participants:

- Andrea Palumbo
- Baptiste Levesque
- Antonio Cardoso

Points abordés :

- Organisation des tâches pour finir le GDD dans les temps

Actions réalisées :

- Relecture du travail effectué jusque là, prise de décision sur certains doutes en suspens (déplacement libre/ sur rail, Monnaie Unique/doubles collectibles etc ...)

Actions à mener pour la prochaine session :

- Finir le GDD/Avancer au maximum

Répartition du travail entre chaque membre du groupe pour la prochaine session :

- Economie -> Antoine
- Résumé du niveaux -> Antonio
- Bestiaire -> Antonio
- Sound design -> Andrea
- UI/UX / Interface utilisateur -> Baptiste
- Vérifier la légende -> Antonio
- Ajouter des schémas -> Antoine & Benoit
- Repasser sur le GDD pour voir si vous êtes raccord avec ce qui est fait et voir s' il manque des choses, reformuler, etc ... -> Antoine & Benoit
- Relecture pour s'assurer que le légendage et le lexique est correct et respecté-> Antonio

Date de la prochaine session :

11/09/2025

Transcript de la réunion :

Réunion #7

Participants:

- Antonio Cardoso
- Lion Clement
- Juan Ospina

Points abordés :

- Récapitulatif du projets dans l'ensemble, et organisations de la soutenances

Actions réalisées :

- réflexion sur la mise en place de l'oral et du doc,
- présentation des avancé des protos
- Retour et conseil sur la suite des proto, prioriser certains axes de développement

Actions à mener pour la prochaine session :

- Finir le doc de présentation, se préparer à la soutenance avec les autres

Répartition du travail entre chaque membre du groupe pour la prochaine session :

Date de la prochaine session :

23/10/2025

Transcript de la réunion :

Date des séances d'échange avec un enseignant :

Séance n°1 : 24 Septembre 2025.

Discussion sur les GDD, définition de priorités, clarification des attendus.

Séance n°2 : 8 Octobre 2025

Point d'avancement, discussions next Steps.

Séance n°3 : 22 Octobre 2025

Point sur les rendus et sur la présentation finale du projet.

Echantillon de captures d'écran échanges entre réunions :

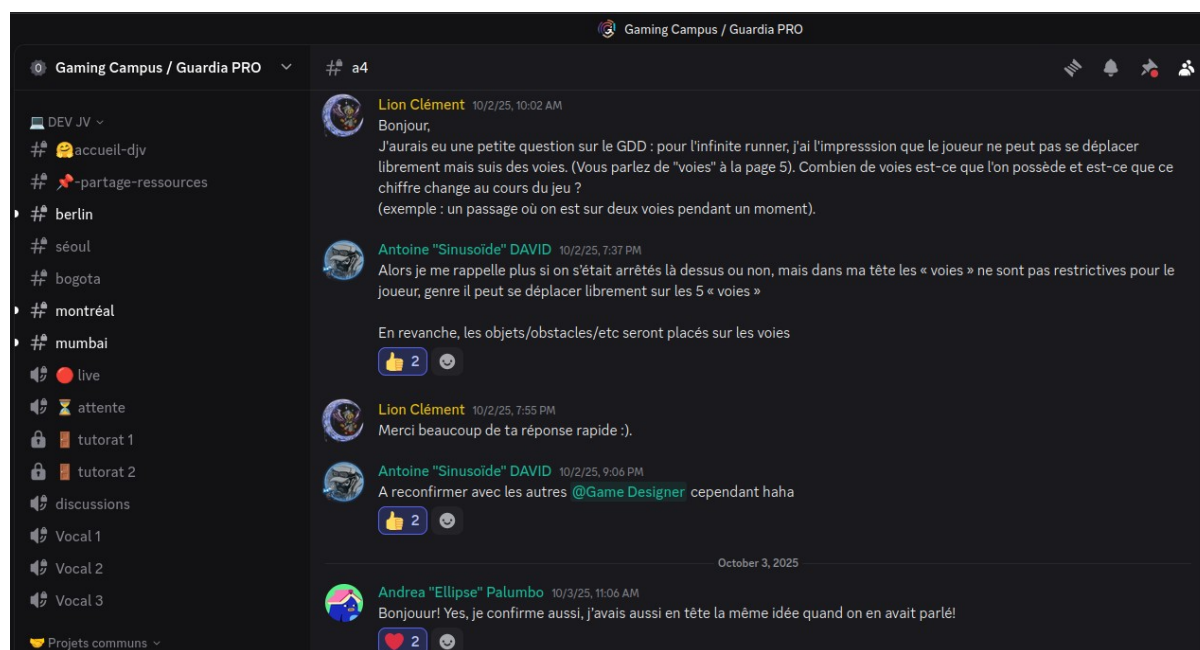


Figure 82: Demande d'éclaircissement sur la notion de lanes : peut-on se déplacer librement ou par colonnes ?

Gaming Campus / Guardia PRO

a4

October 7, 2025

Lion Clément 10/7/25, 8:16 PM

Hello !

Petite question sur les powerup des différents états. La durée d'effet est de 10s. Qu'est-ce que cela veut dire ? Par exemple quand notre oeuf utilise son bouclier coquille, ça veut dire que pendant 10 s il va être capable de bloquer un choc ou alors qu'il peut prendre une blessure et qu'il lui faut 10s de cooldown à partir du moment où il a pris un choc pour soigner son bouclier coquille ?

Même question sur les autres techniques (ralenti temporel dure 10s mais quel est le cooldown ?)

Passez une excellente journée !

Pouvoirs (Temporaires liés à la forme):

Durée d'effet 10s

- Œuf : Bouclier Coquille : immunité à 1 obstacle, la coquille se fissure et disparaît.
- Poussin : Ralentit temporel : ralentit brièvement la vitesse du jeu
- Poulet : Jet d'œuf : lance 3 œufs explosifs pour détruire les obstacles devant soi.
- Coq : Cri du Coq : onde sonore qui repousse obstacles destructible de 2m et Immobilise les obstacles en mouvements.
- Super Coq : Souffle de feu : crache du feu nettoyant la ligne devant lui de tous les obstacle destructible sur quelques mètres
- Dinosaure : Écrasement sismique : smash frontal qui pulvérise tous les obstacles destructible visible à l'écran

October 8, 2025

Antoine "Sinusoïde" DAVID 10/8/25, 8:50 AM

Yo Clément !

Tu as raison c'est mal expliqué, on aurait dû préciser pour chaque pouvoir.

Pour le Bouclier : à l'activation, pendant les 10 prochaines secondes, seul UN obstacle peut être bloqué, ensuite le pouvoir disparaît. Et il me semble qu'on avait dit qu'il fallait ramasser un item pour récupérer le pouvoir (un collectible comme les grains mais pour récupérer une charge de pouvoir, @Game Designer corrigez moi si je me trompe ou si ça a changé)

Pour le ralenti : 10s me paraît énorme pour ralentir le jeu, ensuite vrai j'aurais peut être plutôt mis 5s max mais l'idée c'est

Figure 83: Demande d'éclaircissements sur les pouvoirs.

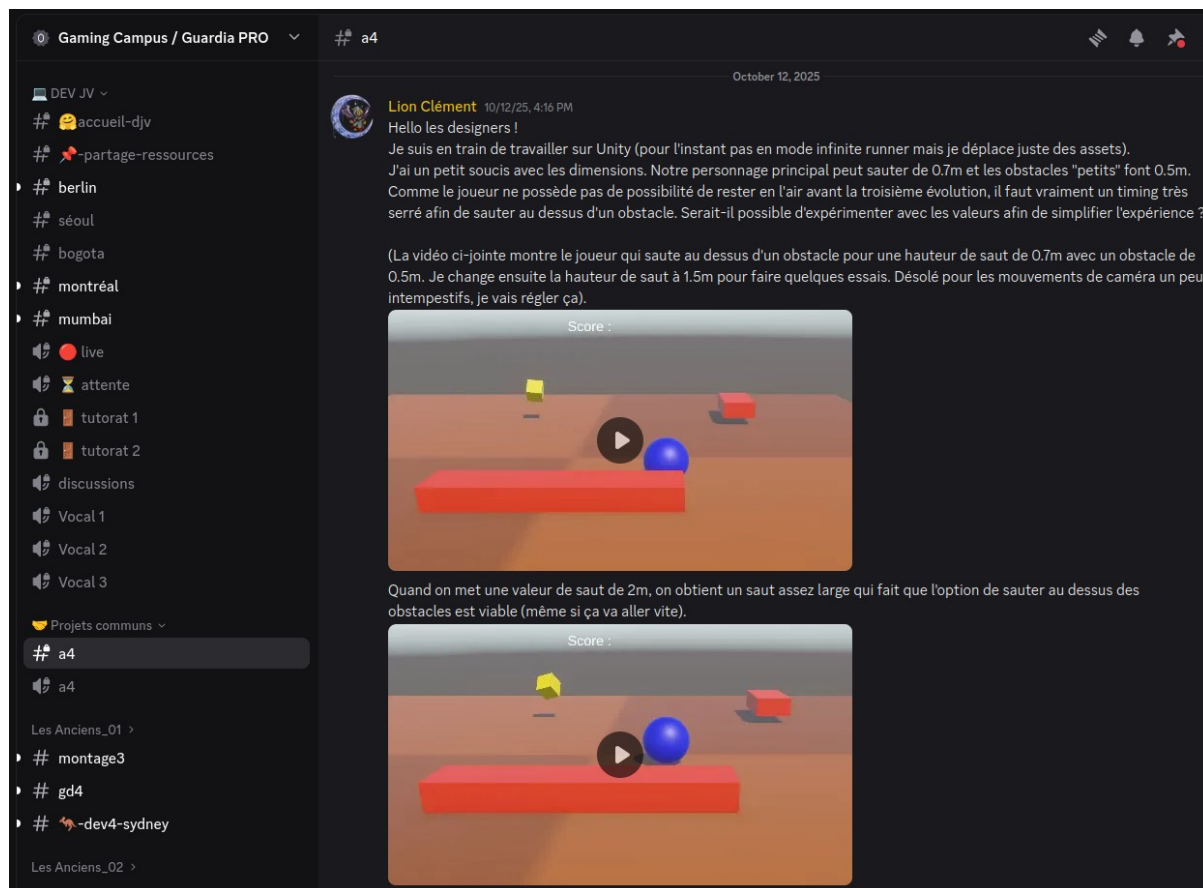


Figure 84: Présentation de premiers résultats et demande de pouvoir modifier les variables précisées dans le GDD pour expérimenter.

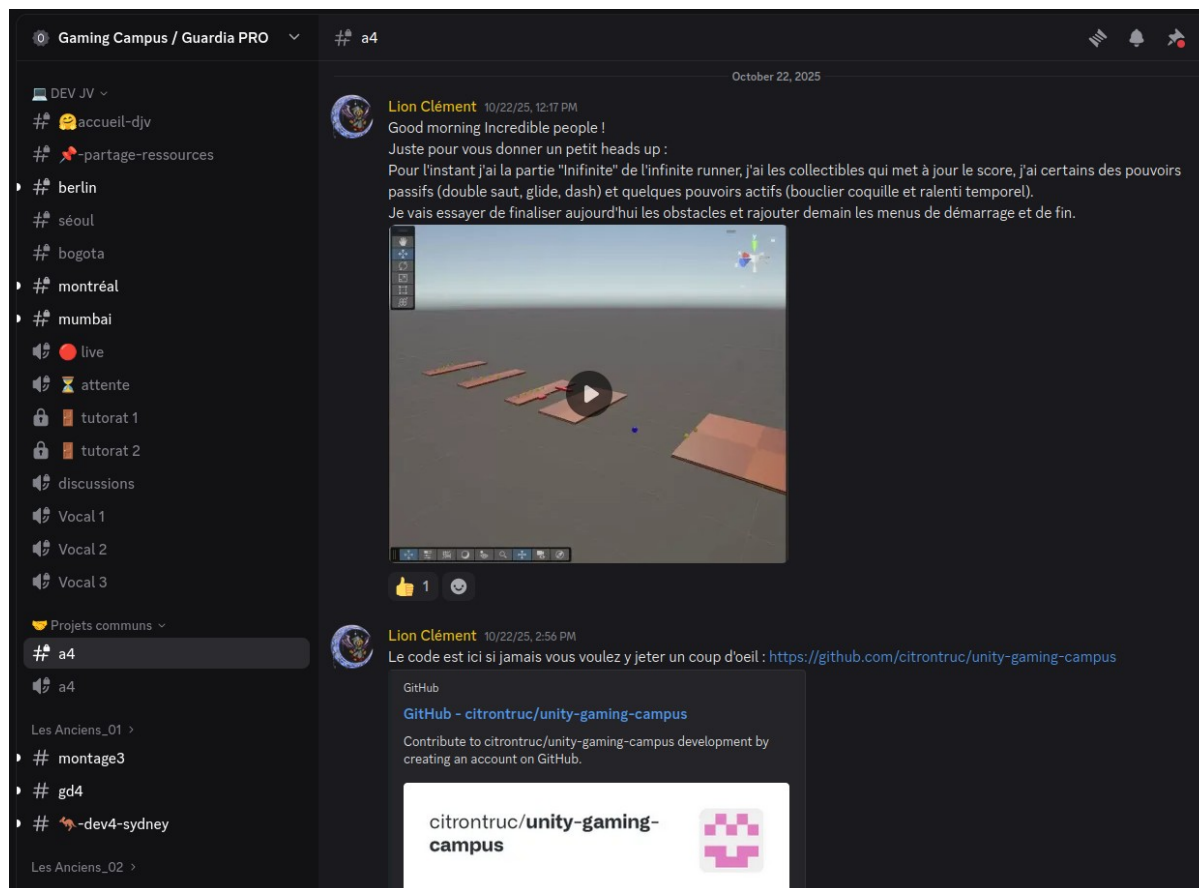


Figure 85: Présentation avancement du projet et partage du code si envie de vérifier le fonctionnement.

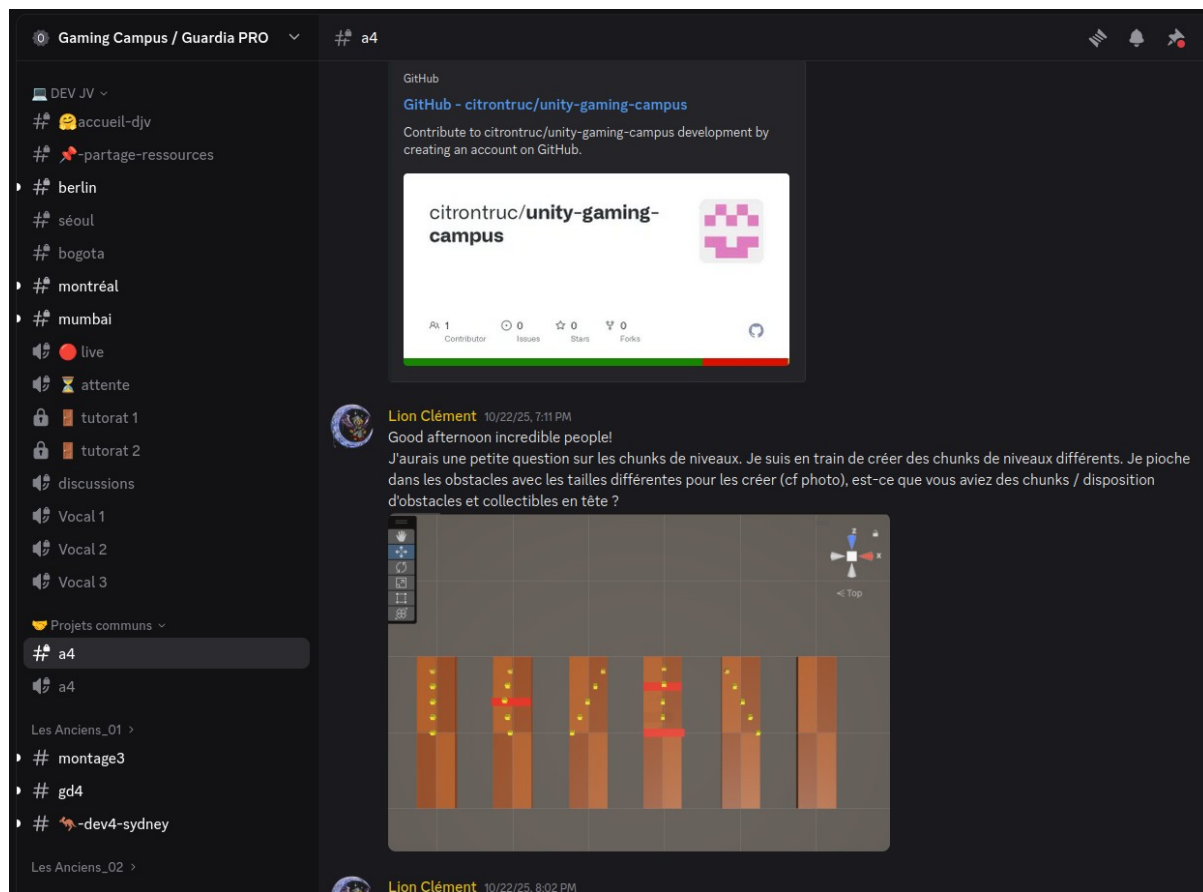


Figure 86: Proposition de premiers chunks de niveau.

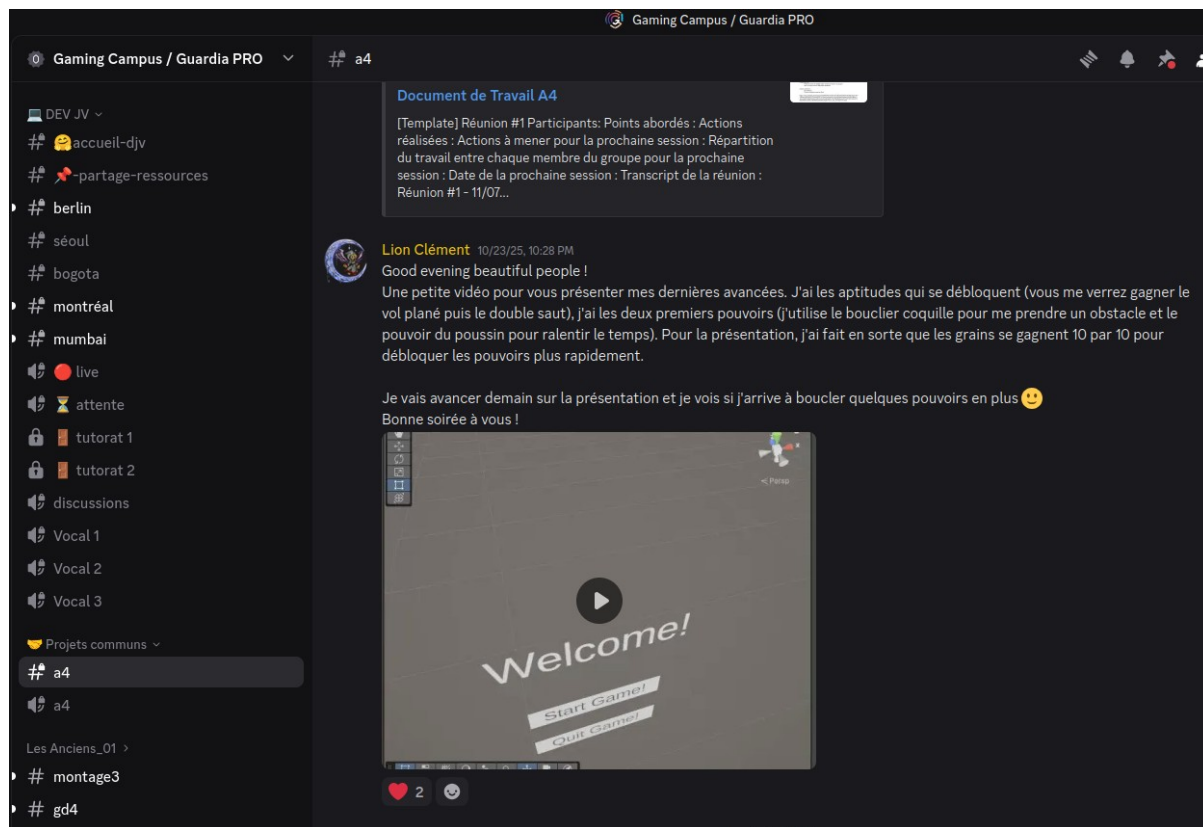


Figure 87: Vidéo de présentation d'une version intermédiaire du jeu.

